PROJECT REPORT

PlaceIQ

Campus Placement & Recruitment Management System: a PostgreSQL database that enforces its own rules

Fundamentals of Database Systems (FDS)

Prof. Debapriyay Mukhopadhyay · PGDBA Batch 12, IIM Calcutta

Member	Name	Presentation segment
M1	Ekta Sharma	Problem & EER
M2	[Member 2]	Schema, normalization & DDL
M3	[Member 3]	SQL queries & rules
M4	[Member 4]	Analytics & performance

November 2026 · github.com/EktaSharma-github/placeiq

Abstract

Campus placement committees typically run recruitment from shared spreadsheets: one row per student, per company, per interview round. Such files duplicate data, suffer update, insertion and deletion anomalies, and cannot enforce policy. PlaceIQ replaces the spreadsheet with a relational database in PostgreSQL 18 whose design follows the full arc of the Fundamentals of Database Systems course. We built an Enhanced ER model using overlapping and disjoint specialization, a weak entity, aggregation and a category (union type); mapped it to 22 relations; and proved every relation to be in Boyce–Codd Normal Form. Fifteen business rules are enforced inside the database by 45 CHECK constraints, 30 foreign keys (several composite), an exclusion constraint, 9 triggers and stored procedures. Seeded mock data for 180 students, 16 companies, 1,184 applications and 1,057 interviews is loaded through those same rules. Five analytical queries demonstrate joins, HAVING, relational division, correlated subqueries and window functions. An analytics layer then asks which students the process serves badly: descriptive statistics and a feature matrix are computed in SQL, and k-means clustering finds 4 student segments that differ sharply in placement outcome (χ^2 p = 0.000011) although outcomes were never clustering inputs. The segments are written back to BCNF tables, and a SQL query turns them into an action list of 52 students. A composite index made a representative query 2,434 times faster on 600,000 rows. A FastAPI web interface, an SQL-injection laboratory sandboxed by a least-privilege role, and an automated suite of 91 tests (all passing) complete the project.

Contents

Abstract 2
1. Introduction 5
  1.1 Background and problem 5
  1.2 Objectives 5
  1.3 Scope and assumptions 5
  1.4 Tools and environment 5
2. Requirements 7
  2.1 Users 7
  2.2 Business rules 7
  2.3 Functional requirements 7
  2.4 Non-functional requirements 8
3. Conceptual design: Enhanced ER model 9
  3.1 Entities and attributes 10
  3.2 Relationships 10
  3.3 Design decisions 10
4. Logical design: relational schema 12
  4.1 Mapping the EER model 12
  4.2 Keys 13
5. Normalization 14

5.1 The starting relation and its anomalies 14
5.2 Functional dependencies 14
5.3 First normal form 15
5.4 Second normal form 15
5.5 Third normal form 15
5.6 Boyce–Codd normal form 15
5.7 Lossless join and dependency preservation 16
5.8 The 3NF-but-not-BCNF case we avoided 16
5.9 Deliberately non-redundant choices 16
6. Physical design and DDL 17
6.1 Data types 17
6.2 Integrity techniques 17
6.3 Referential actions 18
6.4 Catalog summary 18
7. Business logic: views, triggers and procedures 19
8. Data generation 21
9. Analytical queries 22
Q1. Which recruiters are the hardest to crack? 22
Q2. Talent match for a newly posted job (relational division) 23
Q3. Final-placement CTC leaderboard by program (window functions) 24
Q4. Placement velocity: day-by-day cumulative progress 25
Q5. Hidden gems: strong academics, lost in the interview room 26
10. Analytics: statistics in SQL and k-means segmentation 28
10.1 Descriptive statistics in SQL 28
10.2 Feature engineering 32
10.3 Statistics before modelling 34
10.4 k-means clustering 34
10.5 Validating the segments against outcomes 36
10.6 Writing the segments back to the database 36
10.7 Choosing a classification threshold 39
10.8 Limitations of the analysis 39
11. Transactions and concurrency 40
11.1 The concurrency scenario 40
11.2 Lock order and deadlock 40
12. Performance and indexing 41
13. Security 42
13.1 SQL injection 42
13.2 Least privilege (DCL) 42

14. Application layer 43
15. Testing summary 45
16. Learnings, limitations and future work 46
16.1 What we learned 46
16.2 Limitations 46
16.3 Future work 46
17. Team contribution 47
References 48
Appendix A. How to run 49
Appendix B. Repository contents 49

(If page numbers are missing, right-click the table and choose Update Field.)

1. Introduction

1.1 Background and problem

Every year the placement committee coordinates hundreds of students, dozens of recruiters and several hiring cycles (summer internships, final placements and lateral hiring). The committee's working tool is a spreadsheet. Because a spreadsheet is a single flat file, the same company details repeat on every applicant's row, nothing prevents an ineligible student from being added, two interviews can be booked into the same slot, and a student who has already accepted an offer can keep interviewing. These are the classic limitations of file-processing systems discussed in Module 1: redundancy and inconsistency, difficulty of access, integrity problems, atomicity problems and concurrent-access anomalies.

1.2 Objectives

- Model the placement domain conceptually with an Enhanced ER diagram that uses every Module 4 construct.
- Map the model to a relational schema and normalize it to BCNF, with explicit functional dependencies and proofs of lossless, dependency-preserving decomposition.
- Enforce the committee's policy inside the database: declaratively where possible, procedurally (triggers) where a rule spans several tables.
- Generate realistic, reproducible data and answer real business questions with advanced SQL.
- Analyse the data: descriptive statistics and feature engineering in SQL, then k-means segmentation validated with statistical tests, with the results stored back in the database.
- Demonstrate transactions, concurrency control, indexing and SQL-injection defence with measurements, not assertions.
- Verify everything with an automated test suite and present it through a working web interface.

1.3 Scope and assumptions

- One institute, three programs (PGDBA, PGP, PGPEX) and three placement cycles for the academic year 2026-27.
- All company and person names are fictional; the dataset is synthetic but statistically plausible.
- The data snapshot is taken on 28 October 2026: summer placements are complete, final placements are in progress, one job is newly posted.
- Authentication of real users is out of scope; the web interface is a local teaching application.

1.4 Tools and environment

Table 1. *Technology stack*

Layer	Technology	Purpose
Database	PostgreSQL 18.6 (pgAdmin, psql)	Schema, constraints, triggers, procedures, views, roles
Scripts	SQL files 02–07, PowerShell run_all.ps1	Repeatable build: DDL → logic → data → queries → demo → analytics

Layer	Technology	Purpose
Analytics	Jupyter, pandas, scikit-learn, SciPy, matplotlib	Statistical tests, k-means, logistic regression, figures
Application	Python 3.12, FastAPI, psycopg 3	REST API using parameterized SQL only
Front end	HTML, CSS, vanilla JavaScript	Single-page UI: dashboard, students, jobs, analytics, labs
Testing	pytest 8, FastAPI TestClient	91 automated tests against the real database
Version control	Git, GitHub	github.com/EktaSharma-github/placeiq

2. Requirements

2.1 Users

User	Needs
Student	See eligible jobs, apply, track interviews and offers, accept or decline an offer
Recruiter	Post jobs with eligibility and skill requirements, shortlist, interview, make offers
Placement committee	Enforce policy, schedule interviews without clashes, monitor progress, report statistics
Sponsors	Companies and alumni who fund scholarships, events and awards for a cycle

2.2 Business rules

The rules below were collected from the committee's policy and drive every constraint in the design. The third column states where each rule is enforced.

Table 2. *Business rules and their enforcement mechanism*

ID	Rule	Enforced by
BR-1	Every student belongs to exactly one program	NOT NULL foreign key student.program_id
BR-2	Roll number and e-mail are unique	UNIQUE constraints (alternate keys)
BR-3	CGPA lies between 0 and 10	CHECK constraint
BR-4	A job is an internship or a full-time role: never both, never neither	Composite FK on (job_id, role_type) for disjointness; deferred constraint trigger for totality
BR-5	The recruiter posting a job works for that company	Composite FK (posted_by, company_id) → recruiter(person_id, company_id)
BR-6	A student applies to a job at most once	UNIQUE (student_id, job_id)
BR-7	Interviews only in a round of the job applied for	Two composite FKs on interview sharing job_id
BR-8	No overlapping interview slots for a student	EXCLUDE USING gist (student_id =, tstzrange &&)
BR-9	An offer needs an application; at most one per application	FK + UNIQUE; trigger requires a live application
BR-10	responded_at is set iff the offer was accepted or declined	CHECK constraint
BR-11	Eligibility to apply: program, CGPA, backlogs, deadline, not opted out, not placed	BEFORE INSERT trigger (spans 4 tables)
BR-12	One accepted offer per student per cycle; accepting withdraws the rest	BEFORE + AFTER triggers, stored procedure, row lock
BR-13	A sponsor is a company or an alumnus (union type)	Discriminator + composite FK
BR-14	Placed status is derived, never typed in	View v_placement_status
BR-15	A student holding an offer cannot be deleted	ON DELETE RESTRICT blocks the cascade

2.3 Functional requirements

1. Maintain programs, cycles, companies, recruiters, students, alumni, committee members and sponsors.
2. Post jobs (internship or full-time) with eligibility per program, required skills and interview rounds.
3. Accept applications only from eligible students, before the deadline.

4. Schedule interview rounds without overlap and record results and scores.
5. Make offers; accept, decline or expire them; apply the one-offer policy automatically.
6. Report placement status, compensation statistics, selectivity and progress over time.

2.4 Non-functional requirements

- **Integrity:** rules hold even if the application layer is bypassed (enforced in the DBMS).
- **Atomicity:** multi-step operations such as accepting an offer either complete fully or not at all.
- **Reproducibility:** the database can be rebuilt to an identical state in about two seconds.
- **Security:** no SQL injection in application queries; least privilege where untrusted input is demonstrated.
- **Performance:** foreign keys indexed; hot lookups supported by composite and partial indexes.

3. Conceptual design: Enhanced ER model

The EER model is presented in two views for readability. Figure 1 covers people, programs and sponsorship; Figure 2 covers the recruitment pipeline. Both use Chen notation: rectangles for entities (double for weak), diamonds for relationships (double for identifying), ovals for attributes, double lines for total participation, and circles for specialization ('o' overlapping, 'd' disjoint, 'U' category).

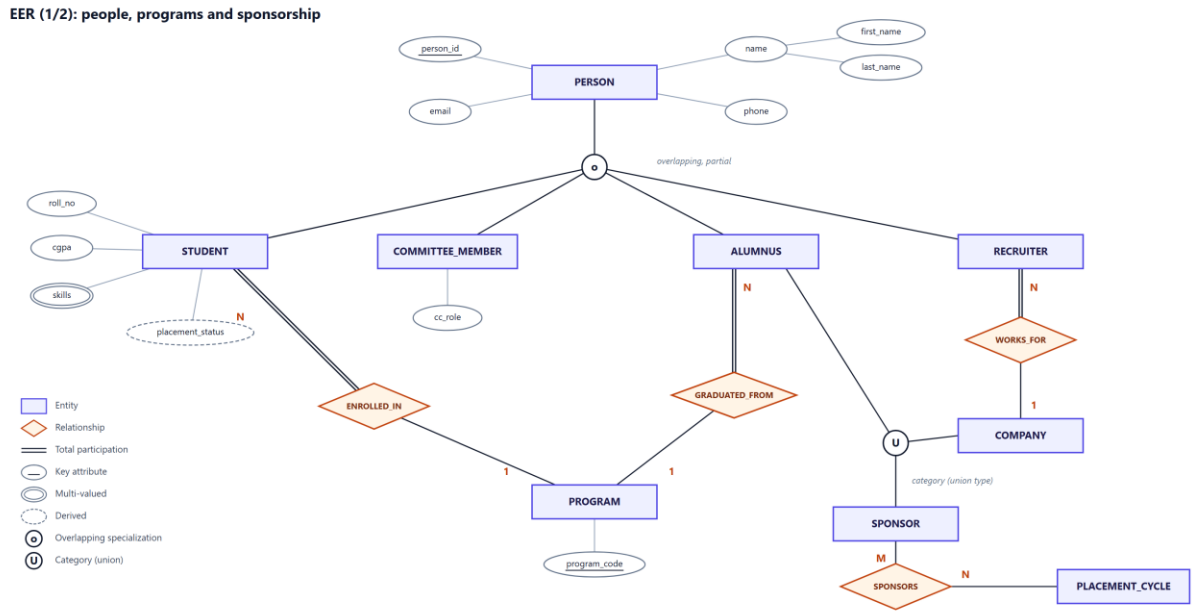


Figure 1. *EER view 1: Person hierarchy (overlapping, partial), programs, and the Sponsor category (union type).*

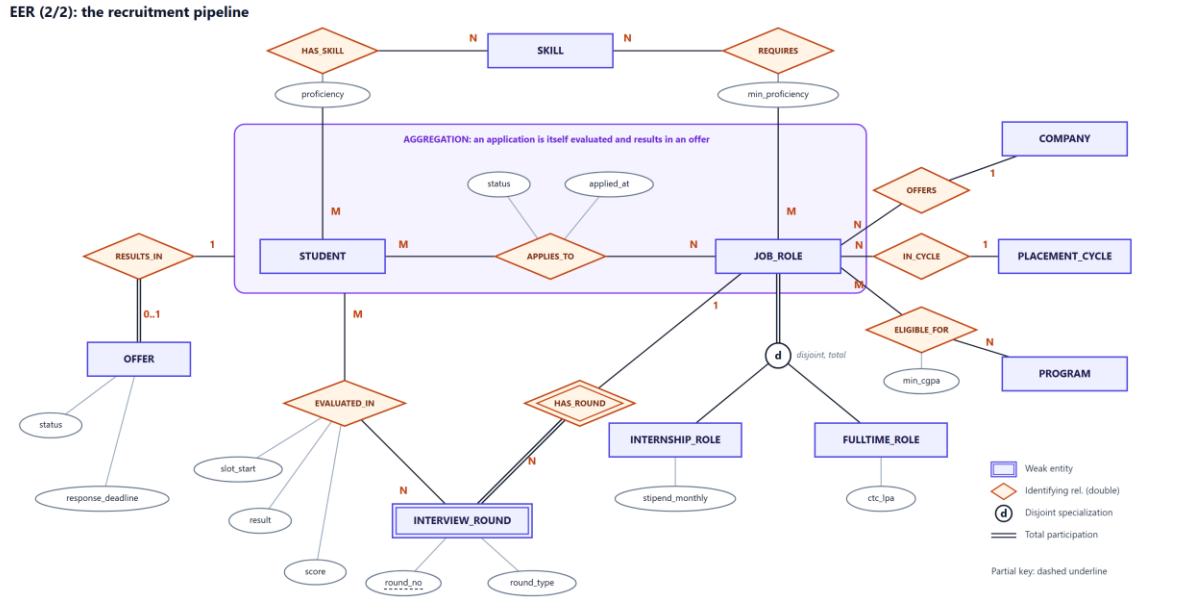


Figure 2. *EER view 2: recruitment pipeline with aggregation, weak entity INTERVIEW_ROUND and disjoint, total job specialization.*

3.1 Entities and attributes

Table 3. *Entities*

Entity	Type	Key and notable attributes
PERSON	Strong, superclass	person_id (key); name (composite: first, last); email (candidate key); phone
STUDENT	Subclass of PERSON	roll_no (candidate key), cgpa, backlogs, opted_out; {skills} multi-valued; placement_status derived
RECRUITER / COMMITTEE_MEMBER / ALUMNUS	Subclasses of PERSON	designation / cc_role, term / graduation_year, current_employer
PROGRAM, COMPANY, PLACEMENT_CYCLE, SKILL	Strong	Surrogate id plus natural candidate key (program_code, company_name, cycle_name, skill_name)
SPONSOR	Category of COMPANY U ALUMNUS	sponsor_id (surrogate), sponsor_type
JOB_ROLE	Strong, superclass	job_id; title, location, openings, deadline
INTERNSHIP_ROLE / FULLTIME_ROLE	Subclasses of JOB_ROLE	stipend_monthly, duration_weeks, ppo_possible / ctc_lpa, fixed_lpa, joining_bonus
INTERVIEW_ROUND	Weak, owner JOB_ROLE	round_no (partial key), round_type, is_elimination
OFFER	Strong (via aggregation)	offered_at, response_deadline, status, responded_at

3.2 Relationships

Table 4. *Relationships with cardinality and participation*

Relationship	Between	Cardinality	Participation / notes
ENROLLED_IN	STUDENT – PROGRAM	N : 1	Student total
GRADUATED_FROM	ALUMNUS – PROGRAM	N : 1	Alumnus total
WORKS_FOR	RECRUITER – COMPANY	N : 1	Recruiter total
OFFERS / IN_CYCLE	COMPANY – JOB_ROLE / JOB_ROLE – CYCLE	1 : N / N : 1	Job total in both
ELIGIBLE_FOR	JOB_ROLE – PROGRAM	M : N	Attributes min_cgpa, max_backlogs
HAS_SKILL / REQUIRES	STUDENT – SKILL / JOB_ROLE – SKILL	M : N	proficiency / min_proficiency
APPLIES_TO	STUDENT – JOB_ROLE	M : N	status, applied_at; treated as an aggregate
HAS_ROUND	JOB_ROLE – INTERVIEW_ROUND	1 : N	Identifying; round total
EVALUATED_IN	[APPLIES_TO] – INTERVIEW_ROUND	M : N	slot, venue, result, score
RESULTS_IN	[APPLIES_TO] – OFFER	1 : 0..1	Offer total
SPONSORS	SPONSOR – PLACEMENT_CYCLE	M : N	purpose, amount

3.3 Design decisions

- **Overlapping, partial specialization of PERSON:** a student may also be a committee member; a guest speaker is only a person.
- **Disjoint, total specialization of JOB_ROLE:** stipend and CTC are different attributes and every job is exactly one kind.

- **Aggregation of APPLIES_TO:** an interview evaluates a specific application (a student for a job), not a student in general; the same applies to an offer. Modelling it this way makes it possible to enforce that a student is interviewed only for a job they applied to.
- **Weak entity INTERVIEW_ROUND:** 'round 2' has no identity outside its job.
- **Category SPONSOR:** companies and alumni are different entity types that play the same sponsoring role.
- **Derived attribute placement_status:** computed from offers, so it can never disagree with them.

4. Logical design: relational schema

4.1 Mapping the EER model

Table 5. *ER-to-relational mapping steps (Elmasri–Navathe)*

Step	Rule	Applied to
1	Strong entity → relation	program, person, company, placement_cycle, job_role, skill
2	Weak entity → relation; PK = owner PK + partial key	interview_round (job_id, round_no)
3	1:1 → FK on the total side + UNIQUE	offer → application (student_id, job_id)
4	1:N → FK on the N side	student.program_id, recruiter.company_id, job_role.company_id, cycle_id, posted_by
5	M:N → bridge relation	eligibility, student_skill, job_required_skill, application, interview, sponsorship
6	Multi-valued attribute → relation	student_skill
8A	Specialization → one relation per subclass sharing the superclass key	student, recruiter, committee_member, alumnus; internship_role, fulltime_role
9	Category → surrogate key relation; FK in each member	sponsor ← company.sponsor_id, alumnus.sponsor_id

Option 8A (a table per subclass) was preferred over a single wide JOB_ROLE table because the single table would need NULLs in half its rows and could not declare that CTC is mandatory for full-time roles with a simple NOT NULL. With 8A every subtype attribute is NOT NULL.

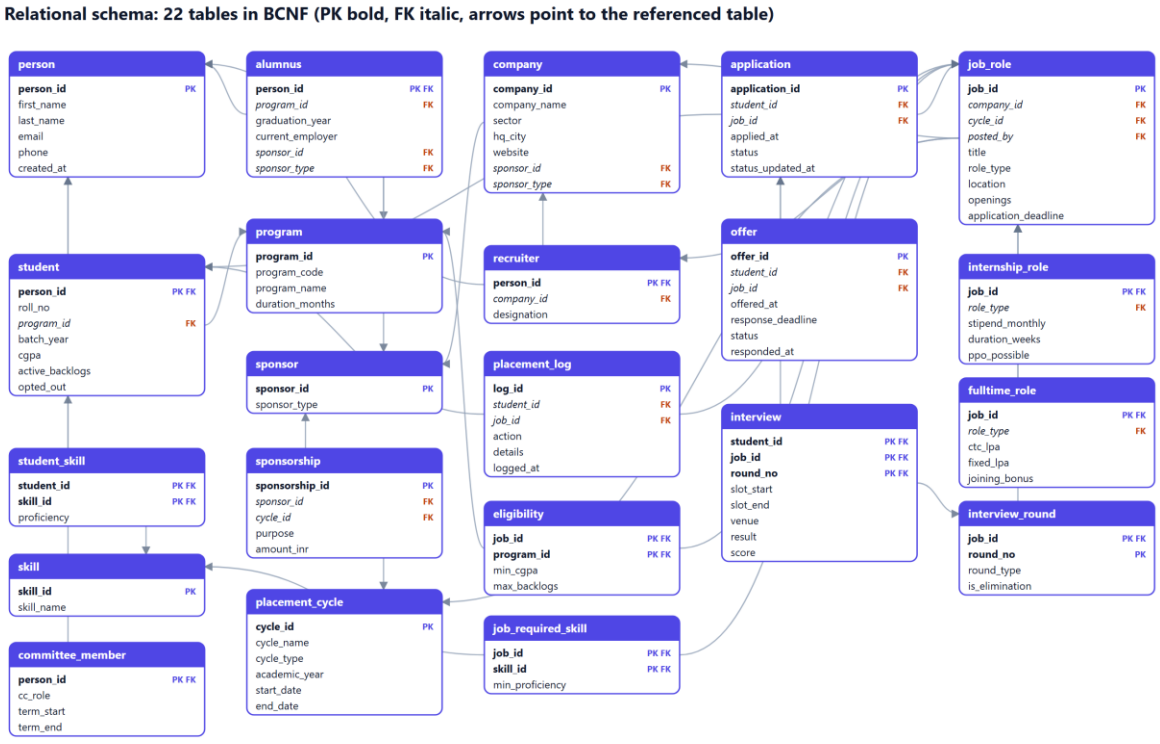


Figure 3. *Relational schema generated from the live catalog: 22 relations; arrows point from a foreign key to the referenced relation.*

4.2 Keys

Table 6. *Keys in the schema*

Key type	Definition	Example in PlaceIQ
Super key	Any attribute set that uniquely identifies a tuple	(person_id, email) in person
Candidate key	A minimal super key	placement_cycle has three: cycle_id; cycle_name; (cycle_type, academic_year)
Primary key	The candidate key chosen for identification	job_id in job_role
Alternate key	A candidate key not chosen as primary, kept UNIQUE	(company_id, cycle_id, title) in job_role; roll_no; email
Foreign key	Attributes referencing a key of another relation	30 in total, e.g. composite (posted_by, company_id) → recruiter

Surrogate keys were introduced after normalization for convenience (short, stable, efficient joins). Every natural key was preserved as a UNIQUE constraint, so no functional dependency is lost by the substitution.

5. Normalization

5.1 The starting relation and its anomalies

The committee's spreadsheet is an unnormalized relation PLACEMENT_SHEET(roll_no, name, email, prog_code, prog_name, cgpa, {skills}, company, sector, hq_city, cycle, job_title, role_type, pay, min_cgpa, round_no, round_type, slot, result, app_status).

- **Update anomaly:** a company changing its headquarters must be edited on every applicant and round row.
- **Insertion anomaly:** a job cannot be recorded before someone applies, because roll_no is part of the key.
- **Deletion anomaly:** deleting the only applicant to a job deletes the job's pay, cut-off and rounds.

5.2 Functional dependencies

Let J denote (company, cycle, job_title), the natural identifier of a job.

Table 7. *Functional dependencies of PLACEMENT_SHEET*

FD	Dependency
FD1	roll_no → name, email, prog_code, cgpa
FD2	email → roll_no
FD3	prog_code → prog_name
FD4	company → sector, hq_city
FD5	J → role_type, pay
FD6	J, prog_code → min_cgpa
FD7	J, round_no → round_type
FD8	roll_no, J → app_status
FD9	roll_no, J, round_no → slot, result
MVD	roll_no →→ skills (independent of every other attribute)

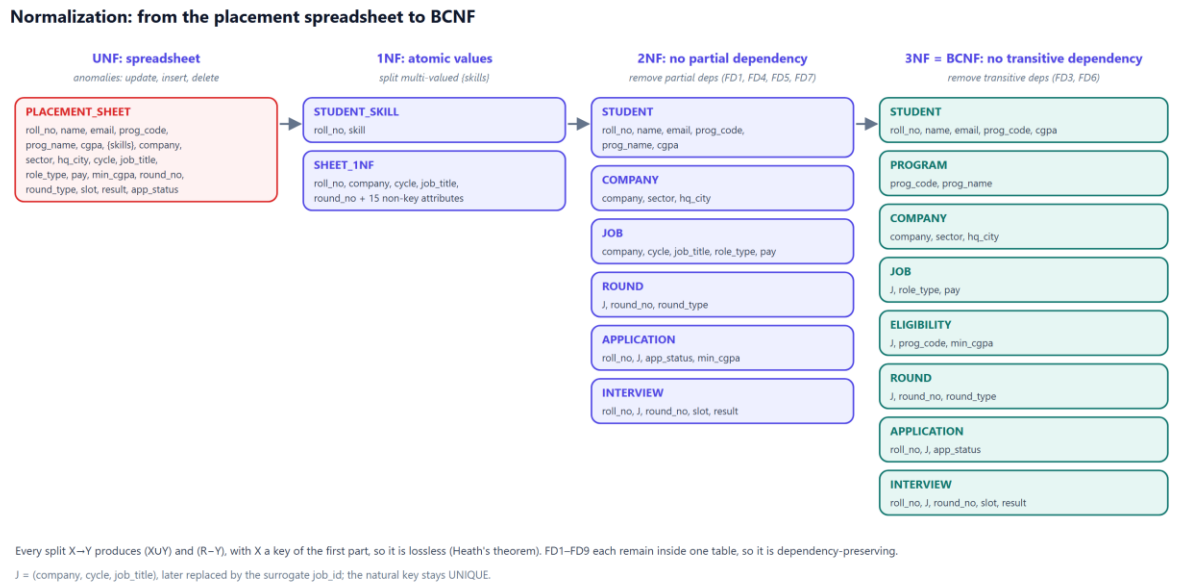


Figure 4. *Decomposition from the spreadsheet to BCNF.*

5.3 First normal form

skills is a list, violating atomicity. Splitting it into one row per skill would multiply rows by the number of rounds and skills (the multi-valued dependency at work), so it moves to StudentSkill(roll_no, skill). The remaining relation has atomic values and key {roll_no, company, cycle, job_title, round_no}.

5.4 Second normal form

Several non-key attributes depend on part of the composite key: student details on roll_no (FD1), company details on company (FD4), job details on J (FD5), round type on (J, round_no) (FD7) and application status on (roll_no, J) (FD8). Each partial dependency becomes its own relation: Student, Company, Job, Round, Application and Interview.

5.5 Third normal form

Two transitive dependencies remain. In Student, roll_no → prog_code → prog_name, so Program(prog_code, prog_name) is separated. In Application, (roll_no, J) → roll_no → prog_code and (J, prog_code) → min_cgpa, so the cut-off moves to Eligibility(J, prog_code, min_cgpa).

5.6 Boyce–Codd normal form

Table 8. *BCNF verification*

Relation	Non-trivial FDs	Every determinant a super key?
Program	prog_code → prog_name	Yes
Student	roll_no → all; email → roll_no	Yes: roll_no and email are both candidate keys
Company	company → sector, hq_city	Yes
Job	J → role_type, pay	Yes
Eligibility	(J, prog_code) → min_cgpa	Yes
Round	(J, round_no) → round_type	Yes

Relation	Non-trivial FDs	Every determinant a super key?
Application	(roll_no, J) → app_status	Yes
Interview	(roll_no, J, round_no) → slot, result	Yes
StudentSkill	none non-trivial (MVD isolated, so also 4NF)	Yes
placement_cycle	each of three candidate keys → all	Yes

5.7 Lossless join and dependency preservation

Each decomposition step split a relation R on a dependency X → Y into (X ∪ Y) and (R − Y), where X is a key of the first part. By Heath's theorem such a binary decomposition is lossless: the natural join of the parts reproduces R exactly. Every dependency FD1–FD9 is contained within a single resulting relation, so the decomposition is also dependency-preserving and every rule can be checked locally.

5.8 The 3NF-but-not-BCNF case we avoided

Design discussion

Suppose each recruiter interviewed for exactly one job. A relation Panel(student, job, recruiter) would have (student, job) → recruiter and recruiter → job.

recruiter is not a super key, so Panel violates BCNF; but job is a prime attribute, so Panel is still in 3NF.

The BCNF decomposition (recruiter, job) + (student, recruiter) is lossless but not dependency-preserving: (student, job) → recruiter can no longer be checked inside one table.

Decision: the business does not need the rule (a recruiter may interview for several roles), so it was not modelled and the schema remains in BCNF without loss.

5.9 Deliberately non-redundant choices

Not stored	Reason
CTC or stipend on offer	offer → job_id → ctc_lpa would be a transitive dependency; the job subtype tables hold pay
placement_status on student	Derived from offers by the view v_placement_status
cycle_id on application	job_id → cycle_id would be transitive; the one-offer-per-cycle rule is a trigger instead

The constant discriminator columns (for example internship_role.role_type = 'INTERNSHIP') are not a normal-form violation: their domain has a single value, so they introduce no non-trivial dependency. They exist only so that composite foreign keys can enforce disjointness declaratively.

6. Physical design and DDL

6.1 Data types

Kind of data	Type chosen	Why
Surrogate keys	INT / SMALLINT GENERATED BY DEFAULT AS IDENTITY	Standard SQL identity columns; small tables use SMALLINT
Money and grades	NUMERIC(p, s)	Exact decimal arithmetic; never floating point
Timestamps	TIMESTAMPTZ	Unambiguous across time zones; required by tstzrange for EXCLUDE
Codes and enumerations	VARCHAR + CHECK (... IN (...))	Readable values with a closed domain
Flags	BOOLEAN	opted_out, ppo_possible, is_elimination

6.2 Integrity techniques

Disjoint specialization with a composite foreign key

```
CREATE TABLE internship_role (
    job_id           INT           PRIMARY KEY,
    role_type        VARCHAR(10)   NOT NULL DEFAULT 'INTERNSHIP' CHECK (role_type =
'INTERNSHIP'),
    stipend_monthly  NUMERIC(10,2) NOT NULL CHECK (stipend_monthly >= 0),
    duration_weeks   SMALLINT      NOT NULL CHECK (duration_weeks BETWEEN 4 AND 26),
    ppo_possible     BOOLEAN       NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_internship_job FOREIGN KEY (job_id, role_type)
        REFERENCES job_role (job_id, role_type) ON DELETE CASCADE
);
```

job_role declares UNIQUE (job_id, role_type). A full-time job has role_type 'FULLTIME', so an internship row for it has nothing to reference and is rejected by the database. Totality (every job has some subtype row) cannot be expressed declaratively and is checked by a deferred constraint trigger at COMMIT.

Interview integrity across the aggregation

```
CONSTRAINT fk_interview_application FOREIGN KEY (student_id, job_id)
    REFERENCES application (student_id, job_id) ON DELETE CASCADE,
CONSTRAINT fk_interview_round FOREIGN KEY (job_id, round_no)
    REFERENCES interview_round (job_id, round_no) ON DELETE CASCADE,
CONSTRAINT excl_student_double_booking
    EXCLUDE USING gist (student_id WITH =, tstzrange(slot_start, slot_end) WITH &&)
```

The two foreign keys share job_id, which guarantees that a student is interviewed only in a round belonging to the job they applied for (BR-7). The exclusion constraint forbids two rows for the same student whose time ranges overlap (BR-8), something no UNIQUE constraint can express; it relies on the btree_gist extension.

Cross-entity consistency

```
CONSTRAINT fk_job_posted_by FOREIGN KEY (posted_by, company_id)
    REFERENCES recruiter (person_id, company_id) ON DELETE SET NULL (posted_by)
```

A recruiter can post a job only for the company they work for (BR-5). The PostgreSQL 15+ column list in SET NULL (posted_by) nulls only the recruiter reference if the recruiter is deleted, keeping the job and its company.

6.3 Referential actions

Action	Used for	Reason
CASCADE	person → subtypes; job_role → rounds and subtypes; application → interviews	The dependent row has no meaning without its owner
RESTRICT	program → student; job_role → application; application → offer	Protect history and formal records
SET NULL (col)	recruiter → job_role.posted_by; sponsor → company/alumnus	Keep the row, forget the link

The RESTRICT-blocks-CASCADE chain (BR-15)

Deleting a person cascades to student and then to application; if an application has an offer, the RESTRICT on offer aborts the entire statement. A student who holds an offer therefore cannot be erased.

6.4 Catalog summary

Table 9. *Objects in the placeiq schema (from the system catalog)*

Object	Count
Tables	22
Primary keys	22
Foreign keys	30
UNIQUE constraints	17
CHECK constraints	45
EXCLUDE constraints	1
Indexes (including key indexes)	55
Triggers	9
Views	2
Stored routines	3

The counts describe the designed schema. The analytics layer (chapter 10) adds 2 tables (segment, student_segment), two foreign keys with their index, and two views (v_student_features, v_segment_profile).

PostgreSQL indexes primary keys and UNIQUE constraints automatically but not foreign-key columns. The DDL therefore creates an index for every foreign key not already covered by the leading columns of another index; test TC-SCH-04 verifies this for all 30 foreign keys.

7. Business logic: views, triggers and procedures

Rules that involve several tables cannot be CHECK constraints, which see only the row being written. These are implemented in PL/pgSQL in 03_logic.sql and are installed before the data is loaded, so the seeded data itself passes them.

Table 10. *Triggers*

Trigger	Timing	Rule
trg_application_eligibility	BEFORE INSERT/UPDATE on application	BR-11: opted out, deadline, program eligible, CGPA cut-off, backlogs, already placed in cycle
trg_offer_requires_open_app	BEFORE INSERT on offer	An offer needs an APPLIED or SHORTLISTED application
trg_offer_marks_app_offered	AFTER INSERT on offer	Application status becomes OFFERED
trg_offer_accept_guard	BEFORE INSERT/UPDATE on offer	BR-12: only pending offers, before the deadline, no second acceptance in the cycle; locks the student row
trg_offer_accept_cascade_upd / _ins	AFTER UPDATE/INSERT on offer	Decline other offers, withdraw open applications, free interview slots, write the audit log
trg_job_total_on_* (3)	Deferred constraint triggers	BR-4 totality: every job has a subtype row at COMMIT

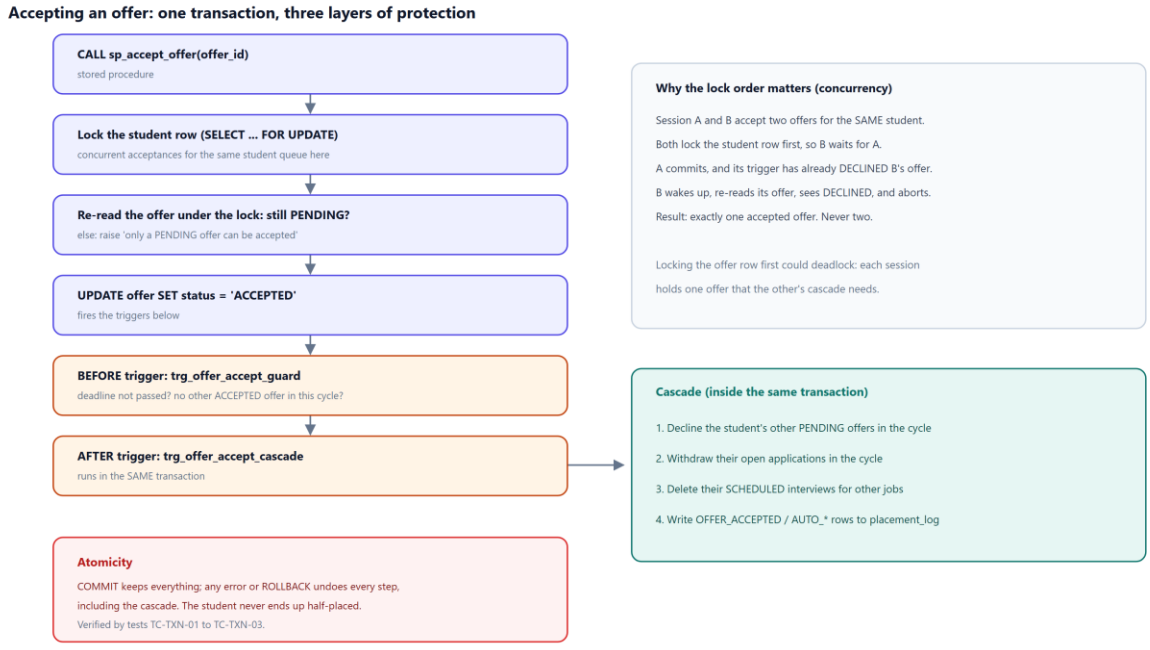


Figure 5. *Accepting an offer: procedure, guard trigger and cascade trigger inside one transaction.*

The cascade (excerpt)

```
WITH withdrawn AS (
    UPDATE application a
       SET status = 'WITHDRAWN',
           status_updated_at = GREATEST(v_when, a.applied_at)
      FROM job_role j
     WHERE j.job_id = a.job_id AND j.cycle_id = v_cycle
       AND a.student_id = NEW.student_id
       AND a.status IN ('APPLIED', 'SHORTLISTED')
    RETURNING a.job_id
```

```
)
INSERT INTO placement_log (student_id, job_id, action, details, logged_at)
SELECT NEW.student_id, w.job_id, 'AUTO_WITHDRAWN', ..., v_when FROM withdrawn w;
```

Table 11. *Views and stored routines*

Object	Purpose
v_job_details	Flattens the job specialization: one row per job with its CTC or stipend (LEFT JOINs to both subtypes)
v_placement_status	BR-14: derived status per (student, cycle): PLACED, OFFER_PENDING, IN_PROCESS, UNPLACED, NOT_APPLIED, OPTED_OUT
fn_apply_for_job(roll_no, job_id)	Parameterized entry point for applying; the trigger validates
sp_accept_offer(offer_id)	Locks the student row first, re-reads the offer, accepts it; the triggers cascade; reports a summary
sp_expire_stale_offers()	Batch job: expires overdue pending offers and logs each

8. Data generation

Reference data (programs, cycles, skills, sixteen fictional companies, recruiters, alumni, jobs, eligibility, required skills) is written as plain INSERT statements so that it can be read on a slide. Students are generated set-based with generate_series; setseed() makes the pseudo-random CGPAs, skills and applications identical on every run. A PL/pgSQL block then simulates recruitment job by job in date order: shortlisting weighted by CGPA, interview rounds with scores, offers to the top performers, and responses. Each acceptance fires the real trigger, which withdraws the student from later jobs in the same cycle.

The whole load is a single transaction. Because the triggers are already installed, every row is validated as it is inserted: if any row broke a rule nothing would be loaded. Identity sequences are advanced with setval() afterwards, because inserting explicit ids does not move them.

Table 12. *Data volumes*

Table	Rows	Table	Rows
student	180	application	1,184
company	16	interview	1,057
job_role	25	offer	96
student_skill	1,146	placement_log	144
person	211	eligibility	54

As of the snapshot, summer placements placed 31 of 148 participating students; final placements have placed 30 of 177 so far, with 10 offers pending and a median accepted CTC of ₹29 LPA (highest ₹38 LPA).

9. Analytical queries

The five queries in 05_queries.sql each answer a question the committee actually asks. Results below were produced on the seeded database.

Q1. Which recruiters are the hardest to crack?

Business question. Among final-placement companies that have finished their process and attracted at least 20 applicants, which ones make offers to a SMALLER share of applicants than the campus-wide average? Students use this to decide where to spend their preparation time.

Concepts. 4-way join through a view, outer join (not every application has an offer), aggregate FILTER, GROUP BY, HAVING that compares a group aggregate with a scalar subquery.

Relational algebra.

```
γ company; count(*)→applicants, count(offer_id)→offers
( Application ⋈ Offer ⋈ JobDetails )
then σ applicants ≥ 20 ∧ offers/applicants < avg_rate
```

```
SELECT d.company_name,
       d.sector,
       count(*)                                                  AS applicants,
       count(*) FILTER (WHERE iv.interviewed)                    AS interviewed,
       count(o.offer_id)                                         AS offers,
       count(o.offer_id) FILTER (WHERE o.status = 'ACCEPTED')    AS accepted,
       round(100.0 * count(o.offer_id) / count(*), 1)            AS offer_rate_pct
FROM application a
JOIN v_job_details d ON d.job_id = a.job_id
LEFT JOIN offer o    ON o.student_id = a.student_id AND o.job_id = a.job_id
LEFT JOIN LATERAL (SELECT TRUE AS interviewed
                   FROM interview i
                   WHERE i.student_id = a.student_id AND i.job_id = a.job_id
                   LIMIT 1) iv ON TRUE
WHERE d.cycle_type = 'FINAL'
  AND EXISTS (SELECT 1 FROM offer x WHERE x.job_id = a.job_id)        -- process complete
GROUP BY d.company_id, d.company_name, d.sector
HAVING count(*) >= 20
   AND count(o.offer_id)::numeric / count(*) <
       (SELECT count(o2.offer_id)::numeric / count(*)                 -- campus-wide offer
rate
        FROM application a2
        JOIN job_role j2 ON j2.job_id = a2.job_id
        LEFT JOIN offer o2 ON o2.student_id = a2.student_id AND o2.job_id = a2.job_id
        WHERE j2.cycle_id = 2
          AND EXISTS (SELECT 1 FROM offer x WHERE x.job_id = a2.job_id))
ORDER BY offer_rate_pct;
```

Table 13. *Q1 result (5 rows)*

company_name	sector	applicants	interviewed	offers	accepted	offer_rate_pct
Meridian Insurance	BFSI	58	22	5	1	8.60
Kestrel Capital	BFSI	42	23	4	4	9.50
Lumen Technologies	TECHNOLOGY	63	27	6	2	9.50
Vertex Bank	BFSI	72	37	7	6	9.70
DataSutra Labs	ANALYTICS	70	30	7	2	10

Q2. Talent match for a newly posted job (relational division)

Business question. Job 25 (Tidewater Logistics, Supply Chain Analytics Lead) has just been posted. Which students are eligible, not yet placed this cycle, and hold EVERY skill the job requires at the required level? The committee will nudge them to apply.

Concepts. Division expressed as "there is NO required skill that the student does NOT have" (double NOT EXISTS, two nested correlated subqueries), plus an anti-join for "not yet placed".

Relational algebra.

```
Qualified(student, skill) = π student,skill (σ proficiency ≥ min (StudentSkill ⋈
JobRequiredSkill_25))
Result = Qualified ÷ π skill (σ job_id = 25 (JobRequiredSkill))
```

```
WITH params AS (SELECT 25 AS job_id)
SELECT s.roll_no,
       concat_ws(' ', p.first_name, p.last_name) AS student_name,
       pr.program_code,
       s.cgpa
FROM params
JOIN eligibility e ON e.job_id = params.job_id
JOIN student s     ON s.program_id = e.program_id
JOIN person p      ON p.person_id = s.person_id
JOIN program pr    ON pr.program_id = s.program_id
WHERE s.cgpa >= e.min_cgpa
  AND s.active_backlogs <= e.max_backlogs
  AND NOT s.opted_out
  -- division: no required skill exists ...
  AND NOT EXISTS (
        SELECT 1
        FROM job_required_skill r
        WHERE r.job_id = params.job_id
          -- ... that this student lacks at the required level
          AND NOT EXISTS (
                SELECT 1
                FROM student_skill ss
                WHERE ss.student_id  = s.person_id
                  AND ss.skill_id    = r.skill_id
                  AND ss.proficiency >= r.min_proficiency))
  -- anti-join: not already placed in this job's cycle
  AND NOT EXISTS (
        SELECT 1
        FROM offer o
        JOIN job_role j  ON j.job_id = o.job_id
        JOIN job_role jt ON jt.job_id = params.job_id
        WHERE o.student_id = s.person_id
          AND o.status = 'ACCEPTED'
          AND j.cycle_id = jt.cycle_id)
ORDER BY s.cgpa DESC;
```

Table 14. *Q2 result (8 rows)*

roll_no	student_name	program_code	cgpa
DBA12-039	Tanvi Jain	PGDBA	9.04
DBA12-059	Kavya Patel	PGDBA	8.40
DBA12-046	Akash Shah	PGDBA	8.01
DBA12-058	Chetan Iyer	PGDBA	7.87
DBA12-036	Ishaan Ghosh	PGDBA	7.70
DBA12-009	Priya Jain	PGDBA	7.44
DBA12-038	Rohan Singh	PGDBA	7.12

roll_no	student_name	program_code	cgpa
PGP62-079	Kavya Verma	PGP	6.99

Q3. Final-placement CTC leaderboard by program (window functions)

Business question. For each program, who are the top 3 earners in final placements so far, how far above their program's average are they, and which pay quartile are they in? This feeds the placement report.

Concepts. Query over a VIEW (v_placement_status, a derived attribute); a named WINDOW with PARTITION BY + ORDER BY; DENSE_RANK (ties share a rank, no gaps), NTILE, AVG() OVER (aggregates without collapsing rows); a subquery to filter on a window result (WHERE cannot see it directly).

```
SELECT program_code, rank_in_program, roll_no, student_name, placed_with, ctc_lpa,
       program_avg_ctc, above_avg_by, pay_quartile, placed_in_program
FROM (
    SELECT program_code,
           roll_no,
           student_name,
           placed_with,
           ctc_lpa,
           DENSE_RANK() OVER w                                                  AS rank_in_program,
           round(AVG(ctc_lpa) OVER (PARTITION BY program_code), 2)       AS program_avg_ctc,
           round(ctc_lpa - AVG(ctc_lpa) OVER (PARTITION BY program_code), 2) AS
above_avg_by,
           NTILE(4) OVER w                                                      AS pay_quartile,
           COUNT(*) OVER (PARTITION BY program_code)                            AS
placed_in_program
    FROM v_placement_status
    WHERE cycle_id = 2 AND placement_status = 'PLACED'
    WINDOW w AS (PARTITION BY program_code ORDER BY ctc_lpa DESC)
) ranked
WHERE rank_in_program <= 3
ORDER BY program_code, rank_in_program, roll_no;
```

Table 15. *Q3 result (14 rows)*

program_code	rank_in_program	roll_no	student_name	placed_with	ctc_lpa	program_avg_ctc	above_avg_by	pay_quartile	placed_in_program
PGDBA	1	DBA12-019	Kavya Verma	Kestrel Capital	38	29.71	8.29	1	12
PGDBA	1	DBA12-026	Pranav Mishra	Kestrel Capital	38	29.71	8.29	1	12
PGDBA	2	DBA12-037	Neha Mukherjee	Northbridge Consulting	34	29.71	4.29	1	12
PGDBA	3	DBA12-003	Pooja Chatterjee	Lumen Technologies	32	29.71	2.29	2	12
PGDBA	3	DBA12-033	Siddharth Chatterjee	Lumen Technologies	32	29.71	2.29	2	12
PGP	1	PGP62-049	Ananya Verma	Kestrel Capital	38	27.78	10.22	1	16
PGP	1	PGP62-063	Pooja Chatterjee	Kestrel Capital	38	27.78	10.22	1	16
PGP	2	PGP62-003	Aditi Chatterjee	Northbridge Consulting	34	27.78	6.22	1	16

program_code	rank_in_program	roll_no	student_name	placed_with	ctc_lpa	program_avg_ctc	above_avg_by	pay_quartile	placed_in_program
PGP	2	PGP62-007	Sanya Mukherjee	Northbridge Consulting	34	27.78	6.22	1	16
PGP	2	PGP62-051	Meera Rao	Northbridge Consulting	34	27.78	6.22	2	16
PGP	3	PGP62-053	Siddharth Kumar	Arcadia Strategy Partners	31.50	27.78	3.72	2	16
PGP	3	PGP62-077	Zoya Gupta	Arcadia Strategy Partners	31.50	27.78	3.72	2	16
PGPEX	1	PGX20-029	Kavya Patel	Arcadia Strategy Partners	31.50	26.75	4.75	1	2
PGPEX	2	PGX20-027	Zoya Sharma	Vertex Bank	22	26.75	-4.75	2	2

Note. DENSE_RANK gives tied CTCs the same rank; NTILE does not. It deals rows into 4 equal buckets, so two students with the same CTC can land in different quartiles. Use PERCENT_RANK() if ties must stay together.

Q4. Placement velocity: day-by-day cumulative progress

Business question. During final placements, how many students from each program accepted an offer each day, and what share of each batch is placed by the end of each day? (The committee's daily progress chart.)

Concepts. Multiple CTEs; generate_series to build a calendar (so days with zero placements still appear); CROSS JOIN calendar x program; LEFT OUTER JOIN to the facts; running total with SUM() OVER (PARTITION BY ... ORDER BY ...).

```
WITH calendar AS (
    SELECT d::date AS day
    FROM generate_series(DATE '2026-10-18', DATE '2026-10-28', INTERVAL '1 day') AS d
),
batch AS (
    SELECT pr.program_id, pr.program_code,
           count(*) FILTER (WHERE NOT s.opted_out) AS batch_size
    FROM program pr JOIN student s ON s.program_id = pr.program_id
    GROUP BY pr.program_id, pr.program_code
),
daily AS (
    SELECT s.program_id,
           (o.responded_at AT TIME ZONE 'Asia/Kolkata')::date AS day,
           count(*) AS accepted
    FROM offer o
    JOIN job_role j ON j.job_id = o.job_id
    JOIN student s  ON s.person_id = o.student_id
    WHERE j.cycle_id = 2 AND o.status = 'ACCEPTED'
    GROUP BY s.program_id, (o.responded_at AT TIME ZONE 'Asia/Kolkata')::date
)
SELECT c.day,
       b.program_code,
       COALESCE(dl.accepted, 0)                                    AS placed_today,
       SUM(COALESCE(dl.accepted, 0)) OVER run                      AS
placed_cumulative,
       b.batch_size,
```

```
       round(100.0 * SUM(COALESCE(dl.accepted, 0)) OVER run / b.batch_size, 1) AS
pct_batch_placed
FROM calendar c
CROSS JOIN batch b
LEFT JOIN daily dl ON dl.program_id = b.program_id AND dl.day = c.day
WINDOW run AS (PARTITION BY b.program_id ORDER BY c.day)
ORDER BY b.program_code, c.day;
```

Table 16. *Q4 result: final day for each program (full output has one row per program per day)*

day	program_code	placed_today	placed_cumulative	batch_size	pct_batch_placed
2026-10-28	PGDBA	0	12	59	20.30
2026-10-28	PGP	4	16	89	18
2026-10-28	PGPEX	0	2	29	6.90

Q5. Hidden gems: strong academics, lost in the interview room

Business question. Which students are still unplaced in final placements even though, at a company that rejected them in an interview round, their CGPA was HIGHER than the average CGPA of the students that company selected? These students need interview coaching, not academic help, and the failed round says which kind.

Concepts. Correlated subquery (re-evaluated for each application row) used both in SELECT and in WHERE; LATERAL join to find the eliminating round; join on a composite key (job_id, round_no); built on a VIEW.

```
SELECT ps.roll_no,
       ps.student_name,
       ps.program_code,
       ps.cgpa,
       d.company_name,
       elim.round_no || ' - ' || elim.round_type || ' (' || elim.result || ')' AS
eliminated_in,
       (SELECT round(avg(s2.cgpa), 2)
        FROM offer o2 JOIN student s2 ON s2.person_id = o2.student_id
        WHERE o2.job_id = a.job_id)                                          AS
selected_avg_cgpa
FROM v_placement_status ps
JOIN application a   ON a.student_id = ps.student_id AND a.status = 'REJECTED'
JOIN v_job_details d ON d.job_id = a.job_id AND d.cycle_id = ps.cycle_id
JOIN LATERAL (
        SELECT i.round_no, r.round_type, i.result
        FROM interview i
        JOIN interview_round r ON r.job_id = i.job_id AND r.round_no = i.round_no
        WHERE i.student_id = a.student_id AND i.job_id = a.job_id
          AND i.result IN ('NOT_CLEARED', 'ABSENT')) elim ON TRUE
WHERE ps.cycle_id = 2
  AND ps.placement_status IN ('UNPLACED', 'IN_PROCESS')
  AND ps.cgpa > (SELECT avg(s2.cgpa)                                         -- correlated on
a.job_id
                 FROM offer o2 JOIN student s2 ON s2.person_id = o2.student_id
                 WHERE o2.job_id = a.job_id)
ORDER BY ps.cgpa DESC, d.company_name;
```

Table 17. *Q5 result (3 rows)*

roll_no	student_name	program_code	cgpa	company_name	eliminated_in	selected_avg_cgpa
DBA12-039	Tanvi Jain	PGDBA	9.04	Helix Analytics	3 - HR (ABSENT)	8.81
DBA12-049	Priya Verma	PGDBA	8.98	Quanta Cloud	1 - APTITUDE (NOT_CLEARED)	8.33

roll_no	student_name	program_code	cgpa	company_name	eliminated_in	selected_avg_cgpa
DBA12-035	Bhavya Sen	PGDBA	8.51	Quanta Cloud	1 - APTITUDE (NOT_CLEARED)	8.33

Q2 was additionally cross-checked against the division-by-counting formulation (GROUP BY … HAVING count(*) = number of required skills) for all 25 jobs; both give identical sets (test TC-QRY-06).

10. Analytics: statistics in SQL and k-means segmentation

Analytics means taking decisions from data. Chapters 3–9 built a database that stores a placement season correctly; this chapter uses it to answer a question the committee cannot answer from a spreadsheet:

> **Analytical question**
> Which kinds of students does the current placement process serve badly, and what should the committee do for each kind?

The workflow follows the order taught in class. Statistics first, then the model. The database does as much of the work as possible, so only a small feature matrix ever leaves it:

1. **Descriptive statistics and cross-tabs in SQL** (queries A1–A5 in 07_analytics.sql).
2. **Feature engineering in SQL:** the view v_student_features turns six normalized tables into one row per student.
3. **Statistical tests** (correlation, ANOVA, point-biserial) to see which features carry signal before any model is fitted.
4. **k-means clustering** in Python (scikit-learn) on standardized profile features; k chosen by elbow and silhouette.
5. **Validation against outcomes** that were never clustering inputs (χ^2 and Kruskal–Wallis tests).
6. **Write-back:** cluster labels are stored in BCNF tables, and SQL queries them again (S1, S2).
7. **Classification threshold:** a logistic model of P(offer) shows the false-positive / false-negative trade-off.

> **Why a DBMS for analytics?**
> An algorithm runs in memory, and loading every raw record for every question does not scale. Most questions need only an aggregate: "how many students with a CGPA of 8–9 were placed?" is one GROUP BY.
>
> In PlaceIQ the DBMS computes all statistics and the whole feature matrix. Python receives 177 rows × 5 features, not the 1,184 applications and 1,057 interviews they summarise.

10.1 Descriptive statistics in SQL

Each query below carries its business question, the course concepts it demonstrates and its relational algebra, exactly as written in 07_analytics.sql.

A1. What does the CGPA distribution look like in each program?

Business question. Recruiters set CGPA cut-offs. Before agreeing to one, the committee needs the centre, spread and quartiles of CGPA for each program.

Concepts. Aggregate functions, ordered-set aggregates (percentile_cont, mode), GROUP BY ROLLUP for a grand-total row.

Relational algebra.

```
γ program_code; count(*), avg(cgpa), stddev(cgpa), ... ( Student ⋈ Program )
```

```
SELECT coalesce(p.program_code, 'ALL')                                          AS program,
       count(*)                                                                 AS n,
       round(avg(s.cgpa), 2)                                                    AS mean,
       round(percentile_cont(0.5) WITHIN GROUP (ORDER BY s.cgpa)::numeric, 2) AS median,
       (mode() WITHIN GROUP (ORDER BY round(s.cgpa * 2) / 2))::numeric(3,1)   AS
mode_half_pt,
       round(stddev_samp(s.cgpa), 2)                                            AS sd,
       min(s.cgpa)                                                              AS min,
       round(percentile_cont(0.25) WITHIN GROUP (ORDER BY s.cgpa)::numeric, 2) AS q1,
       round(percentile_cont(0.75) WITHIN GROUP (ORDER BY s.cgpa)::numeric, 2) AS q3,
       max(s.cgpa)                                                              AS max
FROM student s
JOIN program p ON p.program_id = s.program_id
GROUP BY ROLLUP (p.program_code)
ORDER BY p.program_code NULLS LAST;
```

Table 18. *A1 result*

program	n	mean	median	mode_half_pt	sd	min	q1	q3	max
PGDBA	60	7.82	7.84	7.50	0.69	6.48	7.31	8.36	9.18
PGP	90	7.90	7.87	8	0.82	6.23	7.36	8.39	9.69
PGPEX	30	7.90	7.86	7.50	0.86	6.26	7.34	8.41	9.49
ALL	180	7.88	7.87	8	0.78	6.23	7.33	8.40	9.69

Reading. Mean ≈ median (7.88 vs 7.87), so CGPA is roughly symmetric. The three programs have almost the same centre and spread; a one-way ANOVA confirms it (F = 0.1, p = 0.907), so program and CGPA carry different information.

A2. Does CGPA drive placement? (CGPA band x outcome)

Business question. "How many students have a CGPA between 7 and 8, and how many of them got placed?" -- for every band. If the placement rate climbs steeply with CGPA, CGPA is a strong feature; if not, something else matters.

Concepts. width_bucket for a histogram, LEFT JOIN to a view (students with no placement row still count), aggregate FILTER, derived percentage.

Relational algebra.

```
γ band; count(*), count(σ status='PLACED')
  ( Student ⋈ σ cycle_type='FINAL' (PlacementStatus) )

WITH final_status AS (
    SELECT v.student_id, v.placement_status, v.ctc_lpa
    FROM v_placement_status v
    JOIN placement_cycle pc ON pc.cycle_id = v.cycle_id
    WHERE pc.cycle_type = 'FINAL'
)
SELECT width_bucket(s.cgpa, 6.0, 10.0, 4)                                AS band_no,
       to_char(6.0 + (width_bucket(s.cgpa, 6.0, 10.0, 4) - 1), 'FM9.0') || ' - ' ||
       to_char(6.0 +  width_bucket(s.cgpa, 6.0, 10.0, 4),      'FM90.0') AS cgpa_band,
       count(*)                                                          AS students,
       count(*) FILTER (WHERE f.placement_status = 'PLACED')             AS placed,
       round(100.0 * count(*) FILTER (WHERE f.placement_status = 'PLACED') / count(*), 1)
                                                                         AS placed_pct,
       round(avg(f.ctc_lpa), 2)                                          AS avg_ctc_lpa
FROM student s
LEFT JOIN final_status f ON f.student_id = s.person_id
WHERE NOT s.opted_out
GROUP BY band_no, cgpa_band
ORDER BY band_no;
```

Table 19. *A2 result*

band_no	cgpa_band	students	placed	placed_pct	avg_ctc_lpa
1	6.0 - 7.0	26	0	0	NULL
2	7.0 - 8.0	79	5	6.30	29.20
3	8.0 - 9.0	54	15	27.80	28
4	9.0 - 10.0	18	10	55.60	28.85

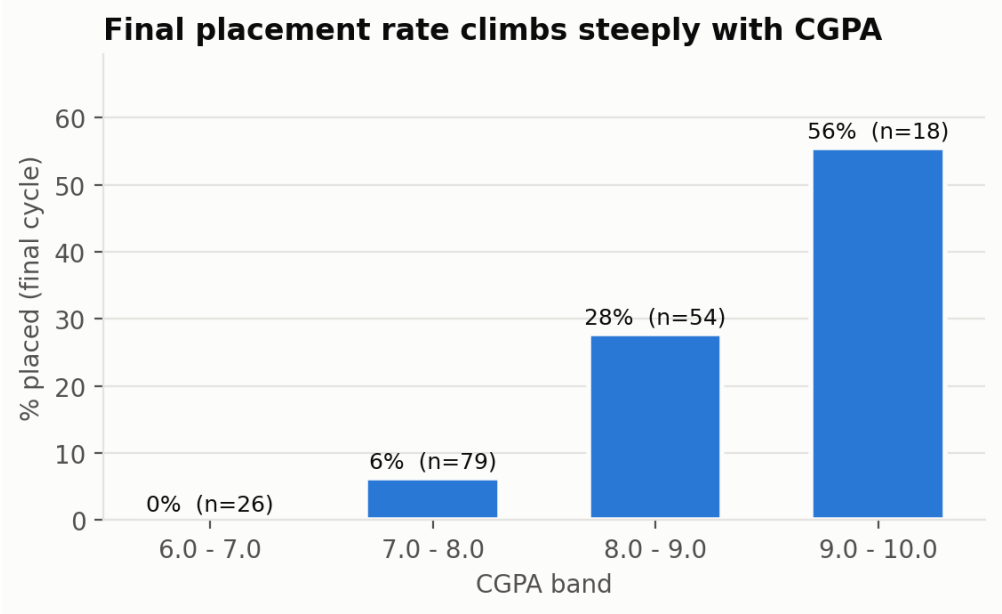


Figure 6. *Final placement rate by CGPA band (query A2).*

Reading. No student below a CGPA of 7 has a final placement yet, against 56% of those above 9. CGPA is a strong feature, but not a sufficient one: most of the 8–9 band is still unplaced.

A3. Where do students drop out of the hiring funnel?

Business question. For each program: of all applications, how many reached an interview, how many got an offer, and how many offers were accepted? A low applied-to-interview ratio points at CVs; a low interview-to-offer ratio points at interview preparation.

Concepts. Conditional aggregation with FILTER, EXISTS inside FILTER, outer join.

Relational algebra.

```
γ program; count(*), count(σ ∃interview), count(offer_id), ...
  ( Application ⋈ Student ⋈ Program ⟕ Offer )
```

```
SELECT p.program_code,
       count(*)                                                    AS applications,
       count(*) FILTER (WHERE EXISTS (SELECT 1 FROM interview i
                                      WHERE i.student_id = a.student_id
                                        AND i.job_id     = a.job_id)) AS interviewed,
       count(o.offer_id)                                           AS offers,
       count(o.offer_id) FILTER (WHERE o.status = 'ACCEPTED')      AS accepted,
       round(100.0 * count(o.offer_id) / count(*), 1)              AS app_to_offer_pct
FROM application a
JOIN student s  ON s.person_id = a.student_id
JOIN program p  ON p.program_id = s.program_id
LEFT JOIN offer o ON o.student_id = a.student_id AND o.job_id = a.job_id
GROUP BY p.program_code
ORDER BY app_to_offer_pct DESC;
```

Table 20. *A3 result*

program_code	applications	interviewed	offers	accepted	app_to_offer_pct
PGP	663	302	61	38	9.20
PGDBA	452	204	32	21	7.10
PGPEX	69	34	3	2	4.30

Reading. About 45% of applications reach an interview in every program, yet only 4–9% become offers: students are lost after the interview, not at CV screening.

A4. Which skills are in short supply?

Business question. For each skill: how many jobs ask for it, and how many students hold it at proficiency 3 or more? Skills with many jobs per qualified student are where the committee should run workshops.

Concepts. Two independent aggregates joined on the key they share, LEFT JOINs so a skill nobody asks for still appears, NULLIF to avoid /0.

Relational algebra.

```
Demand = γ skill_id; count(job_id) (JobRequiredSkill)
Supply = γ skill_id; count(student_id) (σ proficiency ≥ 3 (StudentSkill))
Result = Skill ⋈ Demand ⋈ Supply
```

```
SELECT sk.skill_name,
       coalesce(d.jobs, 0)                                          AS jobs_requiring,
       coalesce(su.students, 0)                                     AS students_proficient,
       round(coalesce(d.jobs, 0)::numeric / NULLIF(su.students, 0), 3) AS jobs_per_student
FROM skill sk
LEFT JOIN (SELECT skill_id, count(*) AS jobs
           FROM job_required_skill GROUP BY skill_id) d  ON d.skill_id  = sk.skill_id
LEFT JOIN (SELECT skill_id, count(*) AS students
           FROM student_skill WHERE proficiency >= 3
           GROUP BY skill_id)                          su ON su.skill_id = sk.skill_id
ORDER BY jobs_per_student DESC NULLS LAST;
```

Table 21. *A4 result*

skill_name	jobs_requiring	students_proficient	jobs_per_student
SQL	11	63	0.17
Excel	13	88	0.15
Statistics	7	50	0.14
Communication	8	79	0.10
Case Solving	6	72	0.08
Python	4	51	0.08
Tableau	2	32	0.06
Machine Learning	2	52	0.04
Product Management	2	75	0.03
Financial Modelling	2	79	0.03
Big Data	1	44	0.02
R	1	59	0.02

Reading. SQL, Excel and Statistics have the most jobs per proficient student, so skill workshops there have the highest return.

A5. What do placed students earn?

Business question. Mean, median and spread of CTC for accepted full-time offers, by program. The median is what the placement brochure should quote, because a few very high offers pull the mean up.

Concepts. Aggregates over a view that hides the specialization join (Job -> Full-time), percentile_cont, stddev, mean vs median.

Relational algebra.

```
γ program; avg(ctc), median(ctc), ...
  ( σ status='ACCEPTED' ∧ role_type='FULLTIME' (Offer ⋈ JobDetails) ⋈ Student ⋈ Program )
```

```
SELECT coalesce(p.program_code, 'ALL')                                          AS program,
       count(*)                                                                 AS
accepted_offers,
       round(avg(d.ctc_lpa), 2)                                                 AS mean_ctc,
       round(percentile_cont(0.5) WITHIN GROUP (ORDER BY d.ctc_lpa)::numeric, 2) AS
median_ctc,
       round(stddev_samp(d.ctc_lpa), 2)                                         AS sd_ctc,
       min(d.ctc_lpa)                                                           AS min_ctc,
       max(d.ctc_lpa)                                                           AS max_ctc
FROM offer o
JOIN v_job_details d ON d.job_id = o.job_id
JOIN student s       ON s.person_id = o.student_id
JOIN program p       ON p.program_id = s.program_id
WHERE o.status = 'ACCEPTED' AND d.role_type = 'FULLTIME'
GROUP BY ROLLUP (p.program_code)
ORDER BY p.program_code NULLS LAST;
```

Table 22. *A5 result*

program	accepted_offers	mean_ctc	median_ctc	sd_ctc	min_ctc	max_ctc
PGDBA	12	29.71	30.25	5.69	22	38
PGP	16	27.78	24.50	6.44	19.50	38
PGPEX	2	26.75	26.75	6.72	22	31.50
ALL	30	28.48	29	6.04	19.50	38

Reading. For PGP the mean CTC (₹27.78 L) is well above the median (₹24.5 L): a few high offers skew the distribution, so the brochure should quote the median.

10.2 Feature engineering

Feature engineering means modelling the problem as data, which needs domain knowledge. The committee can see five things about a student early in the season:

Table 23. *Profile features (clustering inputs)*

Feature	Why it should matter	Built from
cgpa	Recruiters' first filter (A2)	student
tech_score	Analytics roles screen on Python, SQL, ML, Statistics, Big Data, Tableau, R	Σ proficiency over technical skills (student_skill ⋈ skill)
biz_score	Consulting and finance roles screen on cases, Excel, communication, financial modelling, product management	Σ proficiency over business skills
applications	No application, no offer	count(application)

Feature	Why it should matter	Built from
interview_rate	Share of applications that reached an interview: a proxy for CV quality	application, interview

Outcomes are deliberately not inputs. offers, accepted_offer and best_ctc_lpa are kept in the view for validation only. Clustering on them would make "students with offers do well" true by construction; instead we cluster on profile and judge the segments on outcome.

The view is a single statement over six tables. Every column is recomputed from the normalized data whenever it is read, so the features can never go stale.

```
CREATE VIEW v_student_features AS
WITH skills AS (
    SELECT ss.student_id,
           sum(ss.proficiency) FILTER (WHERE sk.skill_name IN
               ('Python', 'SQL', 'R', 'Machine Learning', 'Statistics', 'Big Data',
'Tableau'))   AS tech_score,
           sum(ss.proficiency) FILTER (WHERE sk.skill_name IN
               ('Excel', 'Financial Modelling', 'Case Solving', 'Communication', 'Product
Management')) AS biz_score
    FROM student_skill ss
    JOIN skill sk ON sk.skill_id = ss.skill_id
    GROUP BY ss.student_id
),
apps AS (
    SELECT a.student_id,
           count(*) AS applications,
           count(*) FILTER (WHERE EXISTS (SELECT 1 FROM interview i
                                          WHERE i.student_id = a.student_id
                                            AND i.job_id     = a.job_id)) AS interviewed
    FROM application a
    GROUP BY a.student_id
),
interviews AS (
    SELECT student_id, avg(score) AS avg_score
    FROM interview
    WHERE score IS NOT NULL
    GROUP BY student_id
),
offers AS (
    SELECT o.student_id,
           count(*)                                    AS offers,
           bool_or(o.status = 'ACCEPTED')              AS accepted_offer,
           max(f.ctc_lpa)                              AS best_ctc_lpa
    FROM offer o
    LEFT JOIN fulltime_role f ON f.job_id = o.job_id
    GROUP BY o.student_id
)
SELECT s.person_id                                                    AS student_id,
       s.roll_no,
       p.program_code,
       -- profile features
       s.cgpa,
       coalesce(sk.tech_score, 0)::int                                AS tech_score,
       coalesce(sk.biz_score, 0)::int                                 AS biz_score,
       coalesce(ap.applications, 0)::int                              AS applications,
       round(coalesce(ap.interviewed::numeric / NULLIF(ap.applications, 0), 0), 3)
                                                                      AS interview_rate,
       -- outcomes
       coalesce(o.offers, 0)::int                                     AS offers,
       coalesce(o.accepted_offer, FALSE)                              AS accepted_offer,
       o.best_ctc_lpa,
       round(iv.avg_score, 1)                                         AS avg_interview_score
FROM student s
JOIN program p          ON p.program_id  = s.program_id
LEFT JOIN skills sk     ON sk.student_id = s.person_id
LEFT JOIN apps ap       ON ap.student_id = s.person_id
```

```
LEFT JOIN interviews iv ON iv.student_id = s.person_id
LEFT JOIN offers o      ON o.student_id  = s.person_id
WHERE NOT s.opted_out;
```

10.3 Statistics before modelling

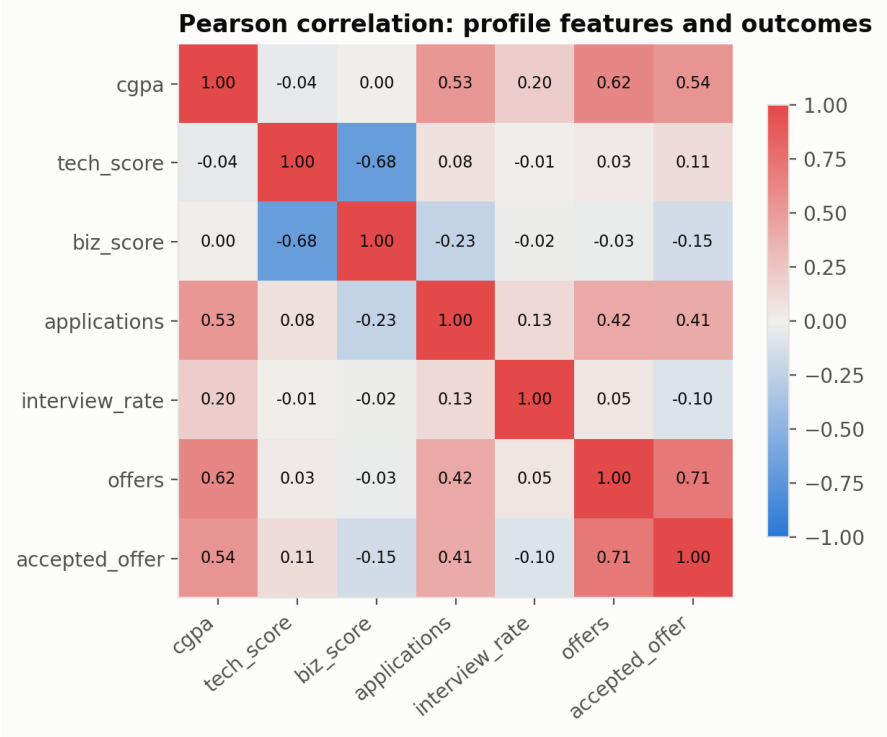


Figure 7. *Pearson correlations between profile features and outcomes.*

Table 24. *Statistical tests*

Test	Hypothesis / measure	Result	Conclusion
Pearson correlation	tech_score vs biz_score	r = -0.68	Students specialise; the main axis the clusters will find
One-way ANOVA	H_0: equal mean CGPA in all programs	F = 0.1, p = 0.907	Not rejected: programs do not differ in CGPA
Point-biserial	accepted_offer vs cgpa	r = 0.54, p = 8.3e-15	Significant association
Point-biserial	accepted_offer vs applications	r = 0.41, p = 1.3e-8	Significant association
Point-biserial	accepted_offer vs interview_rate	r = -0.1, p = 0.19	Not significant on its own

10.4 k-means clustering

Scaling. k-means assigns points to the nearest centroid by Euclidean distance. Unscaled, applications (0–14) would dominate interview_rate (0–1), so every feature is standardized to mean 0 and standard deviation 1 (StandardScaler).

Algorithm. k-means++ initialisation, 20 restarts, fixed random seed (reproducible). For each k it minimises the within-cluster sum of squares (inertia).

Choosing k. Inertia always falls as k grows, so we look for the elbow and check it against the silhouette score: how much closer each point is to its own cluster than to the next, from −1 to 1.

Table 25. *k scan*

k	Inertia	Mean silhouette
2	638.6	0.276
3	484.9	0.283
4	405.1	0.298
5	361.2	0.269
6	325.1	0.23
7	296.6	0.228
8	273.9	0.232

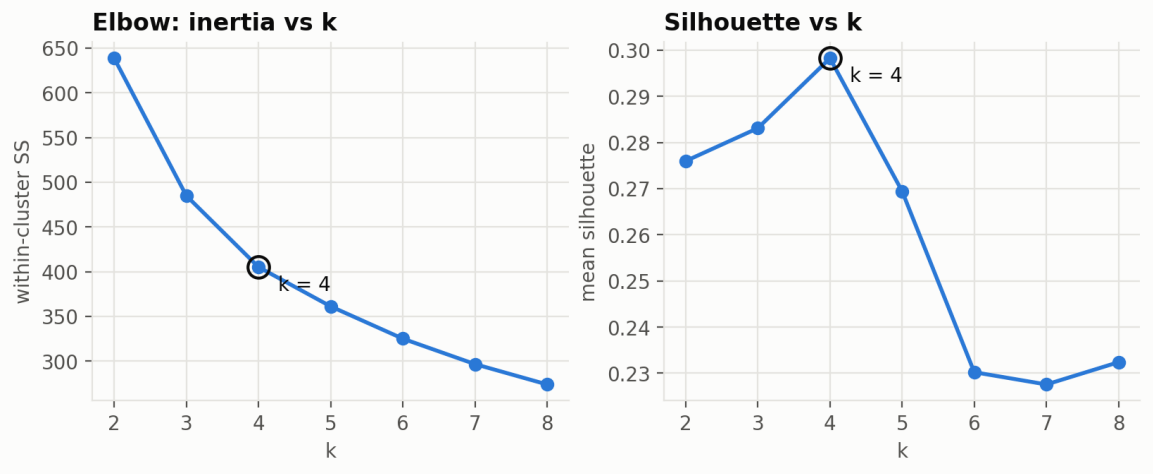


Figure 8. *Elbow and silhouette: both point to k = 4.*

k = 4 gives the highest silhouette (0.298) and sits at the elbow. A silhouette near 0.3 means the segments overlap somewhat, which is normal for data about people. The segments are named from their centroids by rule (most technical; then most applications; then highest interview rate), so the names do not depend on the arbitrary cluster numbering.

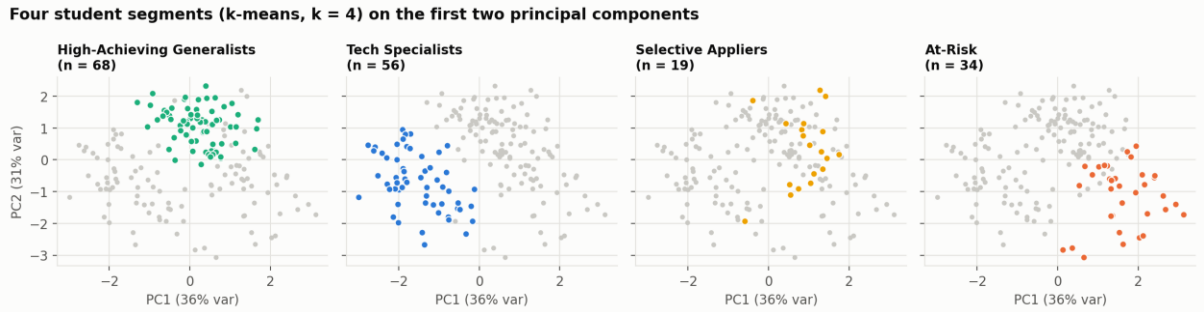


Figure 9. *The segments on the first two principal components (35.9% and 30.7% of variance), one panel per segment.*

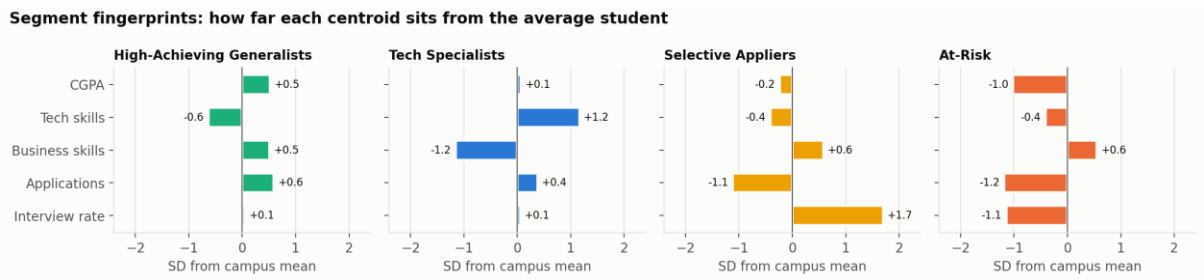


Figure 10. *Segment fingerprints: each centroid in standard deviations from the campus mean.*

Table 26. *Segment profiles (view v_segment_profile)*

Segment	n	CGPA	Tech	Biz	Apps	Interview rate	Offers	Accepted %	Best CTC
High-Achieving Generalists	68	8.27	5	13.7	8.9	0.45	0.87	41.2	27
Tech Specialists	56	7.92	17.8	3.8	8.1	0.45	0.57	35.7	29.30
Selective Appliers	19	7.68	6.6	14.1	2.6	0.88	0.21	0	25.75
At-Risk	34	7.08	6.6	13.9	2.3	0.14	0.03	2.9	NULL

10.5 Validating the segments against outcomes

The segments were built without any outcome data. If they still differ in outcomes, the profile features capture something real. A χ^2 test of independence between segment and accepted offer gives $\chi^2 = 25.7$ (dof 3, $p = 0.000011$); a Kruskal–Wallis test on the number of offers gives $H = 30.9$ ($p = 9e\text{-}7$). Both reject independence decisively.

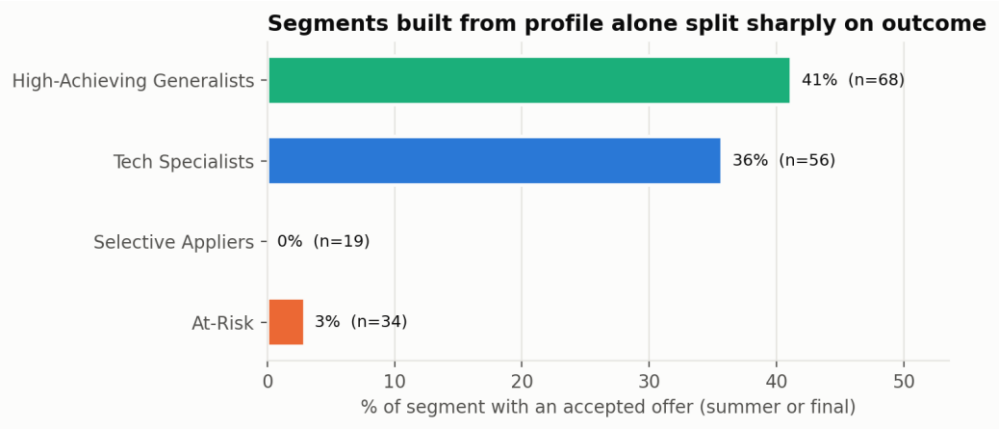


Figure 11. *Share of each segment with an accepted offer.*

Table 27. *Interpretation and recommended action*

Segment	Who they are	Recommended action
High-Achieving Generalists	High CGPA (8.27), business skills, about 9 applications each; 41% accepted	Nothing urgent; help them choose between offers
Tech Specialists	Mostly PGDBA, strong technical skills; 36% accepted, highest CTC (₹29.3 L)	Steer towards analytics roles where A4 shows demand
Selective Appliers	Average CGPA, 88% reach interview, but only 2.6 applications; 0% accepted	Apply more widely: their CVs work, their reach does not
At-Risk	Lowest CGPA (7.08), few applications, rarely shortlisted; 3% accepted	Priority: CV review, then mock interviews

The non-obvious finding is the Selective Appliers. Their CGPA is near the average, so no CGPA cut-off would flag them; the clustering found them through behaviour.

10.6 Writing the segments back to the database

The labels are stored in the database, not in a CSV on one laptop, so every other query can join on them. Two tables keep the design in BCNF: segment → segment_name would be a dependency on a non-key attribute if the name were stored with each student. The foreign keys guarantee that every label belongs to a real student and a real segment, and the foreign key is indexed like every other in the schema.

```
CREATE TABLE IF NOT EXISTS segment (
    segment      smallint    PRIMARY KEY CHECK (segment >= 0),
    segment_name varchar(40) NOT NULL UNIQUE,
    description  text        NOT NULL
);

CREATE TABLE IF NOT EXISTS student_segment (
    student_id   integer     PRIMARY KEY REFERENCES student (person_id) ON DELETE CASCADE,
    segment      smallint    NOT NULL REFERENCES segment (segment) ON DELETE CASCADE,
    assigned_at  timestamptz NOT NULL DEFAULT now()
);

-- Every FK gets an index (project convention, test TC-SCH-04): joins on
-- segment and ON DELETE CASCADE from SEGMENT must not scan the whole table.
CREATE INDEX IF NOT EXISTS idx_student_segment_segment ON student_segment (segment);
```

The notebook replaces the whole segmentation in one transaction (DELETE both tables, INSERT the new rows, COMMIT). Either every student gets the new label or nothing changes, so a failed run can never leave a mixture of old and new labels.

S1. How is each program spread across the segments?

Business question. Are segments just programs in disguise, or does each program contain several kinds of students? Mixed programs mean a program-level policy (one workshop for all PGP students) misses part of the cohort.

Concepts. Join of a table written by an external tool, GROUP BY two columns, window function (share within program).

Relational algebra.

```
γ program_code, segment_name; count(*)
  ( Segment ⋈ StudentSegment ⋈ StudentFeatures )

SELECT f.program_code,
       sg.segment_name,
       count(*)                                                                    AS students,
       round(100.0 * count(*) / sum(count(*)) OVER (PARTITION BY f.program_code), 1) AS
pct_of_program
FROM student_segment g
JOIN segment sg           ON sg.segment = g.segment
JOIN v_student_features f ON f.student_id = g.student_id
GROUP BY f.program_code, sg.segment_name
ORDER BY f.program_code, students DESC;
```

Table 28. *S1 result*

program_code	segment_name	students	pct_of_program
PGDBA	Tech Specialists	54	91.50
PGDBA	At-Risk	3	5.10
PGDBA	High-Achieving Generalists	1	1.70
PGDBA	Selective Appliers	1	1.70
PGP	High-Achieving Generalists	62	69.70
PGP	At-Risk	16	18
PGP	Selective Appliers	9	10.10
PGP	Tech Specialists	2	2.20
PGPEX	At-Risk	15	51.70
PGPEX	Selective Appliers	9	31

program_code	segment_name	students	pct_of_program
PGPEX	High-Achieving Generalists	5	17.20

Reading. PGDBA is almost a single segment, but PGP and PGPEX each spread across three. A program-wide policy ("one workshop for all PGP students") would miss most of their at-risk students.

S2. Who needs help now?

Business question. Students without an accepted offer in every segment whose acceptance rate is below the campus average, with a suggested action, so the committee can contact them this week.

Concepts. Nested subqueries (IN over a view, compared with a scalar aggregate), join with the segment lookup, CASE to derive a recommendation.

Relational algebra.

```
Weak = π segment ( σ accepted_pct < avg_rate (SegmentProfile) )
π roll_no, ... ( σ ¬accepted_offer ( StudentSegment ⋉ Weak ⋈ Segment ⋈ StudentFeatures ) )
```

```
SELECT sg.segment_name,
       f.roll_no,
       f.program_code,
       f.cgpa,
       f.tech_score,
       f.biz_score,
       f.applications,
       f.interview_rate,
       CASE
           WHEN f.applications < 3          THEN 'Apply to more roles'
           WHEN f.interview_rate < 0.4      THEN 'CV review'
           WHEN f.tech_score < f.biz_score  THEN 'Technical skills workshop'
           ELSE                                  'Mock interviews'
       END AS suggested_action
FROM student_segment g
JOIN segment sg           ON sg.segment = g.segment
JOIN v_student_features f ON f.student_id = g.student_id
WHERE NOT f.accepted_offer
  AND g.segment IN (SELECT segment FROM v_segment_profile
                    WHERE accepted_pct < (SELECT 100.0 * avg(accepted_offer::int)
                                          FROM v_student_features))
ORDER BY sg.segment_name, f.applications, f.cgpa;
```

Table 29. *S2 result, summarised (52 students)*

Segment	Suggested action	Students
At-Risk	Apply to more roles	19
At-Risk	CV review	12
At-Risk	Technical skills workshop	2
Selective Appliers	Apply to more roles	10
Selective Appliers	Technical skills workshop	8
Selective Appliers	Mock interviews	1

Reading. The query turns the analysis into a weekly to-do list for the committee. More than half of the 52 students simply need to apply to more roles, which costs the committee only a reminder.

10.7 Choosing a classification threshold

A classifier outputs a probability. Someone must choose the threshold t that turns it into a decision. Here a logistic regression estimates P(offer | profile) with 5-fold cross-validation, so every student is scored by a model that never saw them. A student is flagged for support if P(offer) < t.

Table 30. *Threshold sweep (positive class: student with no offer)*

t	Flagged	TP	FP	FN	TN	Precision	Recall
0.1	43	42	1	69	65	0.98	0.38
0.2	67	63	4	48	62	0.94	0.57
0.3	86	78	8	33	58	0.91	0.7
0.4	97	83	14	28	52	0.86	0.75
0.5	107	89	18	22	48	0.83	0.8
0.6	127	99	28	12	38	0.78	0.89
0.7	137	104	33	7	33	0.76	0.94
0.8	152	108	44	3	22	0.71	0.97
0.9	163	111	52	0	14	0.68	1

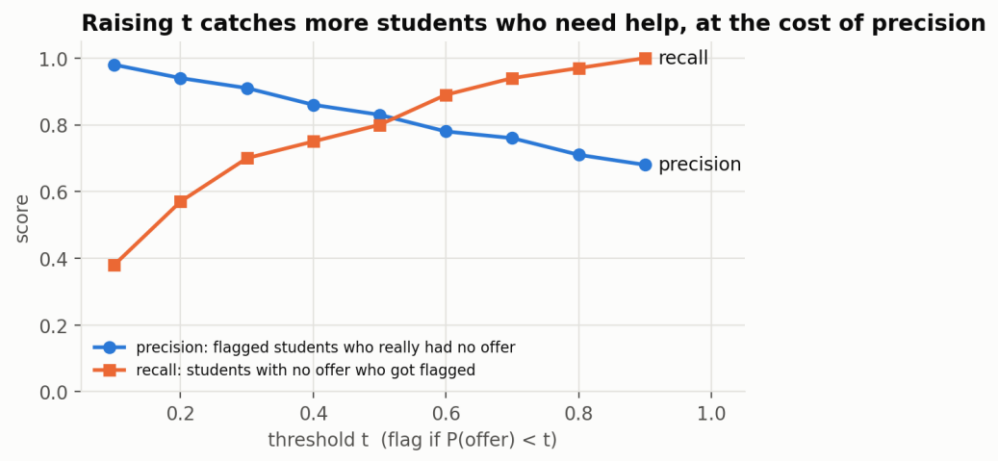


Figure 12. *Precision and recall as the threshold changes.*

A low t misses students who end up with no offer (false negatives). A high t spends workshop seats on students who would have been fine (false positives). A missed student costs more, so a threshold around 0.6 is defensible: it catches 89% of the students with no offer at 78% precision. The standardized coefficients agree with the clusters: CGPA (1.68) and applications (0.71) dominate.

10.8 Limitations of the analysis

- The data is simulated by 04_data.sql, so effect sizes reflect how the simulation was written. The method, not the numbers, is what transfers to a real season.
- The final cycle is in progress; outcomes are a snapshot and will change.
- A silhouette of 0.298 means the segment boundaries are soft: borderline students deserve individual attention.
- Segments are recomputed by running the notebook; a database rebuild empties the segment tables until it is run again.

11. Transactions and concurrency

Table 31. *ACID properties in PlaceIQ*

Property	How the project demonstrates it	Test
Atomicity	Accepting an offer and its whole cascade is one transaction; a ROLLBACK restores offers, applications, interviews and the log	TC-TXN-01
Consistency	Constraints at statement end; deferred totality trigger at COMMIT; a job without a subtype cannot commit	TC-TXN-04, TC-CON-05
Isolation	Concurrent acceptances for one student are serialized by a row lock on the student	TC-TXN-02, 03
Durability	After COMMIT returns, PostgreSQL's write-ahead log preserves the change across a crash	(engine guarantee)

11.1 The concurrency scenario

Under the default READ COMMITTED isolation level, two sessions accepting two different offers for the same student could both pass the 'no other accepted offer' check before either commits, leaving the student with two accepted offers. sp_accept_offer therefore takes SELECT … FOR UPDATE on the student's row before touching any offer. Session B blocks until session A commits; it then re-reads its offer, finds that A's cascade has already declined it, and aborts. The automated test runs this with two real connections and a thread, and asserts that exactly one accepted offer remains.

11.2 Lock order and deadlock

An earlier version locked the offer row first. Two concurrent acceptances could then each hold one offer while their cascades needed the other's, a classic deadlock (PostgreSQL would detect it and abort one session). Locking the common parent row, the student, first imposes a single global lock order, which eliminates the cycle.

12. Performance and indexing

The live tables are small enough that sequential scans are optimal, so indexing was measured on a 600,000-row application archive built inside a transaction. Each figure is the best of three EXPLAIN ANALYZE runs recorded by the performance tests.

Table 32. *Index measurements (query: open applications for one job)*

Configuration	Plan	Execution time	Index size
No index	Sequential scan of 600,000 rows	60.839 ms	—
B-tree (job_id, status)	Bitmap index scan	0.025 ms	4,488 KB
Partial (job_id) WHERE status IN ('APPLIED','SHORTLISTED')	Bitmap index scan	0.018 ms	176 KB

The composite index is about 2,434 times faster than the sequential scan. The partial index, covering only the roughly 4% of rows that are still open, is 26 times smaller than the full index and faster still. A partial index is used in production too: idx_offer_accepted_student supports the triggers' 'has an accepted offer?' lookups.

Finding: the leading-column rule on PostgreSQL 18

We expected a filter on status alone to ignore the (job_id, status) index. PostgreSQL 18 did use it, but by reading the entire index: 4,383 blocks, against 3 blocks for a proper seek on job_id.

The accurate statement of the rule is that an index cannot be used to seek on a non-leading column; at best the whole index (or the whole table) is read. Our test and demo script were corrected accordingly.

13. Security

13.1 SQL injection

Every query in the application passes user input as a bound parameter (psycopg %s placeholders, sent separately from the SQL text as $1, $2 …), so input can never change the structure of a statement. To demonstrate the difference, the web interface includes a laboratory that runs the same student search two ways.

Input	String concatenation	Bound parameter
Sen	6 rows	6 rows
x' OR '1'='1	180 rows: every student leaked	0 rows
x' UNION SELECT roll_no, cgpa::text, phone …	180 rows of CGPAs and phone numbers	0 rows
x'; DROP TABLE student; --	Refused: SQLSTATE 25006 (read-only transaction)	0 rows

13.2 Least privilege (DCL)

```
CREATE ROLE placeiq_readonly NOLOGIN;
GRANT USAGE  ON SCHEMA placeiq TO placeiq_readonly;
GRANT SELECT ON placeiq.student, placeiq.person TO placeiq_readonly;
```

The deliberately vulnerable path runs as this role, inside a READ ONLY transaction with a two-second statement timeout, and is always rolled back. Even a successful injection can therefore only read two tables: writes fail, other tables are denied (SQLSTATE 42501) and superuser functions such as pg_read_file are unavailable. The web server binds to 127.0.0.1 only.

14. Application layer

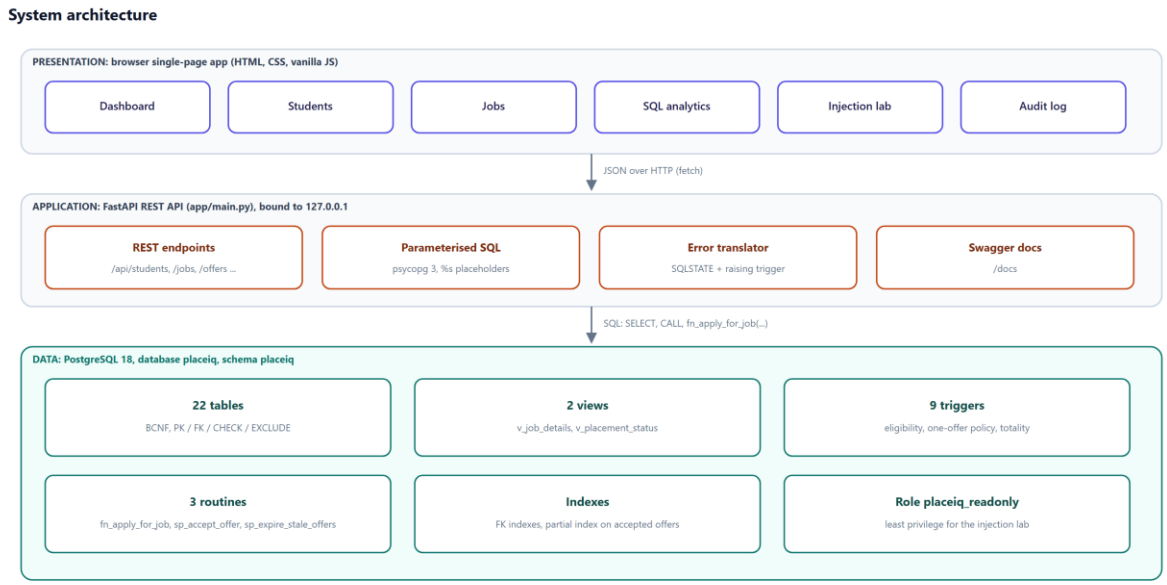


Figure 13. *Three-tier architecture: browser single-page app, FastAPI REST API, PostgreSQL.*

The REST API (app/main.py) is a thin layer: it calls the views, the stored function and the procedures, and translates database errors into JSON containing PostgreSQL's message, the SQLSTATE and the name of the trigger function that raised it. The browser shows that explanation directly, so users see why the database refused an action.

Table 33. *Main REST endpoints*

Method and path	Purpose
GET /api/summary, /api/velocity	Dashboard KPIs, status by program, cumulative placement series
GET /api/students, /api/students/{roll}	Search; full profile with applications, interviews, skills, audit trail
POST /api/applications	Apply (fn_apply_for_job; eligibility trigger)
POST /api/offers, /api/offers/{id}/accept | decline	Recruiter offer; acceptance via sp_accept_offer with the cascade returned
GET /api/jobs, /api/jobs/{id}	Jobs, funnel, rounds, applicants, talent match (Query 2)
GET/POST /api/queries, /api/queries/{n}/run	The five queries, parsed from 05_queries.sql, run read-only
GET /api/security/search	SQL-injection laboratory (sandboxed)
POST /api/admin/reset	Rebuild the seeded database

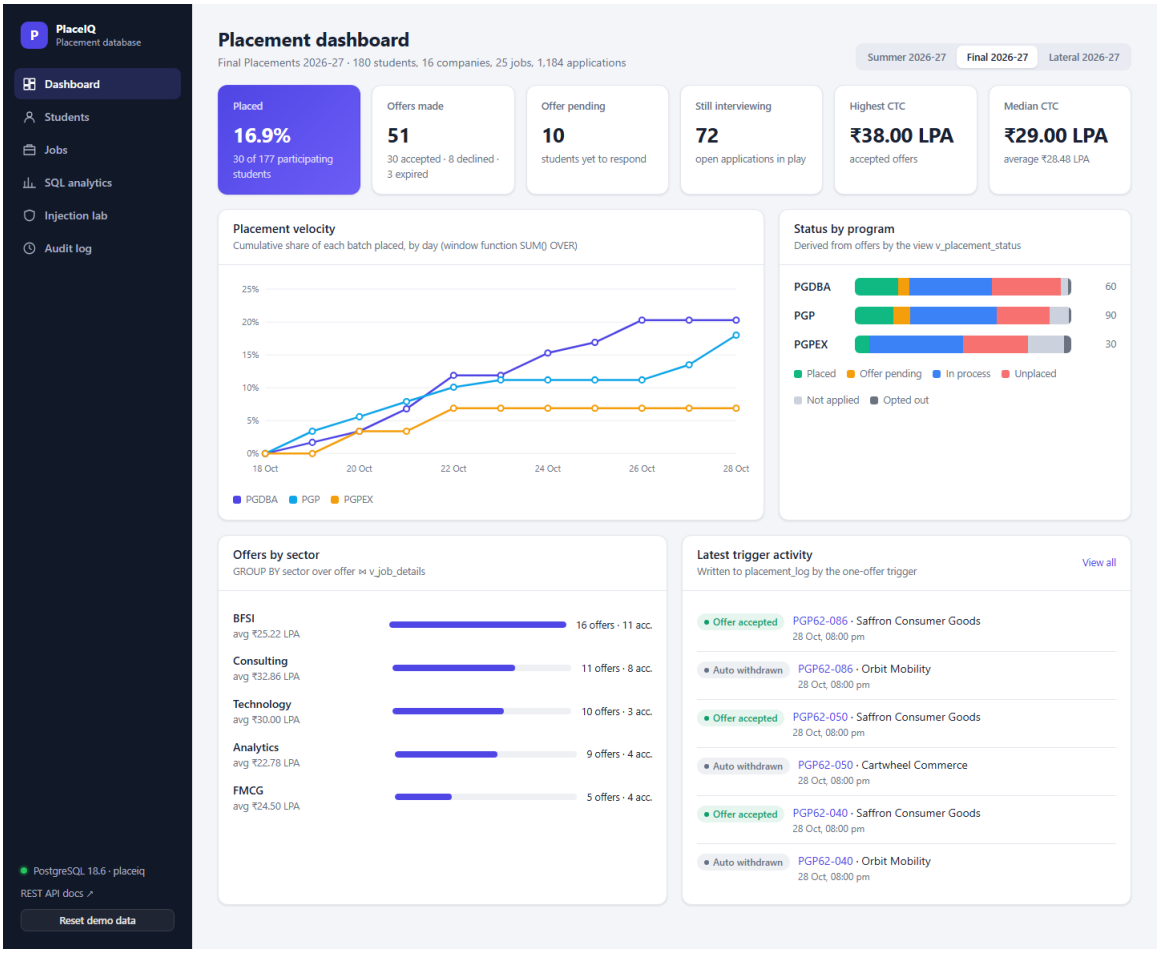


Figure 14. *Dashboard: KPIs, placement velocity (window function) and status by program (view).*

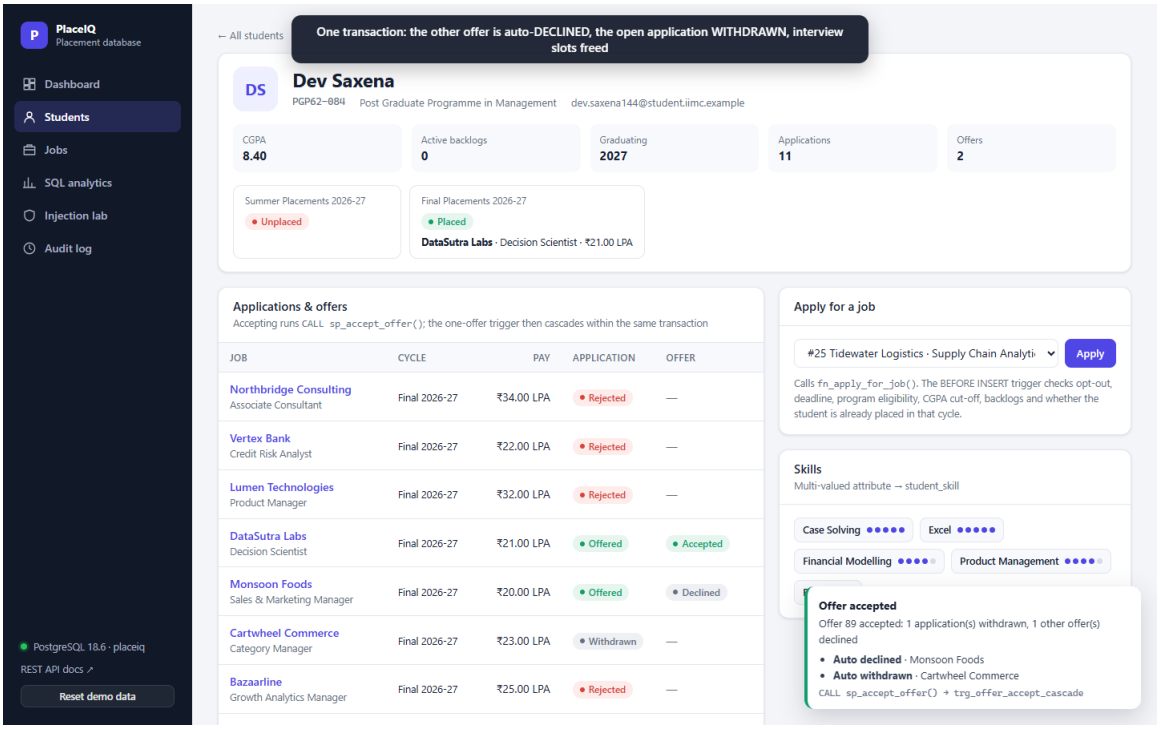


Figure 15. *Student profile after accepting an offer: the cascade declined one offer and withdrew one application.*

15. Testing summary

The automated suite (tests/) contains 91 test cases that run against the real database with pytest; all 91 pass. Each test runs in a transaction that is rolled back, the database is rebuilt at the start and end of a run, and tests that must commit (concurrency, API writes) restore the seed afterwards. Full details, the traceability matrix and the defect log are in the separate Testing Report.

Module	Tests	Result
api	11	All passed
constraints	20	All passed
data	9	All passed
performance	3	All passed
queries	9	All passed
schema	10	All passed
security	9	All passed
transactions	4	All passed
triggers	16	All passed

16. Learnings, limitations and future work

16.1 What we learned

- **Modelling decides what can be enforced.** Treating the application as an aggregation is what made it possible to guarantee, with plain foreign keys, that interviews and offers belong to a real application.
- **Normalization is a proof, not a feeling.** Writing every functional dependency exposed the transitive dependency hidden in min_cgpa and justified the Eligibility table.
- **Declare before you code.** CHECK constraints, composite foreign keys and an exclusion constraint reject bad data even when the application is bypassed; triggers are for what cannot be declared.
- **Concurrency must be tested, not assumed.** A two-connection test showed why lock order matters and turned a possible deadlock into a clean queue.
- **Measure performance.** The leading-column rule behaved differently on PostgreSQL 18 than the textbook summary suggested; measuring blocks read gave the accurate explanation.
- **Let the DBMS do the first pass of analytics.** Statistics, cross-tabs and the feature matrix are SQL; only a small matrix reached Python.
- **Cluster on profile, judge on outcome.** Keeping outcomes out of the clustering made the χ^2 validation meaningful, and surfaced the Selective Appliers, whom a CGPA cut-off would never flag.

16.2 Limitations

- Synthetic data: distributions are plausible but not drawn from a real placement season, so the analytics findings illustrate the method rather than a real campus.
- No user authentication or row-level security per student or recruiter.
- The UI is a teaching interface, tested end to end through the recorded demo rather than a browser test suite.

16.3 Future work

- Row-level security policies so that each student sees only their own rows.
- SERIALIZABLE isolation as an alternative to explicit locking, with retry logic.
- Table partitioning of the application archive by academic year.
- Materialized views for the dashboard with scheduled refresh.

17. Team contribution

Member	Name	Owned in the project and the presentation
M1	Ekta Sharma	Problem analysis, business rules, EER model (both views), Chen diagrams
M2	[Member 2]	ER-to-relational mapping, keys, functional dependencies, normalization to BCNF and proofs, DDL and integrity techniques
M3	[Member 3]	Data generation, the five analytical queries, triggers and procedures, transactions and concurrency, live demo
M4	[Member 4]	Analytics (SQL statistics, feature engineering, k-means, validation, write-back), indexing, security, testing

All members reviewed each other's sections; the presentation follows the same split.

References

1. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill Education, 2019.
2. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.
3. PostgreSQL Global Development Group, PostgreSQL 18 Documentation: constraints, triggers, transaction isolation, indexes, EXPLAIN.
4. NPTEL (IIT Madras), Database Management Systems: ER model, relational model, SQL, relational schema design, storage and indexing.
5. Course material, Fundamentals of Database Systems, PGDBA Batch 12: data models, relational algebra, SQL, query optimization, SQL injection, REST and databases.

Appendix A. How to run

```
# 1. Database (PostgreSQL 15+; from the Project folder)
.\run_all.ps1 -Password <postgres-password>

# 2. Queries and the scripted demo
psql -U postgres -h localhost -d placeiq -f 05_queries.sql
psql -U postgres -h localhost -d placeiq -f 06_demo.sql      # ends in ROLLBACK

# 3. Web UI
cd app
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python main.py                                 # http://127.0.0.1:8000

# 4. Tests (pip install pytest httpx into the same venv)
app\.venv\Scripts\python -m pytest tests -v

# 5. Analytics (after any rebuild; fills the segment tables)
cd analytics
jupyter nbconvert --to notebook --execute --inplace placeiq_kmeans.ipynb
```

Appendix B. Repository contents

Path	Contents
01_design.md	Design notes: rules, EER, mapping, normalization
02_ddl.sql	Schema
03_logic.sql	Views, triggers, procedures, role
04_data.sql	Seeded data and recruitment simulation
05_queries.sql	The five queries
06_demo.sql	Scripted psql demo
07_analytics.sql	SQL statistics, feature view, segment tables and queries
analytics/	k-means notebook (also as HTML) and figures
app/	FastAPI backend and web UI
tests/	pytest suite and results
deliverables/	Report, guide, testing report, runbook, slides, video, diagrams